

武汉大学

本科毕业论文（设计）

DTMB 外源雷达参考信号重构方法及
硬件实现

姓 名：陈 泽 远
学 号：2022302121363
专 业：通信工程
学 院：电子信息学院
指导教师：万 显 荣

二〇二六年 四月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名： 指导教师签名：

日 期： 年 月 日

版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名： 指导教师签名：

日 期： 年 月 日

摘 要

外辐射源雷达是一种利用第三方非合作辐射源进行目标探测的双/多基地雷达系统，具有隐蔽性强、抗干扰、低成本等优势。我国自主知识产权的数字地面多媒体广播（DTMB）采用时域同步正交频分复用（TDS-OFDM）技术，覆盖广泛、抗多径能力强，带宽稳定、模糊函数近似“图钉”状，可利用其作为外辐射源雷达系统的照射源。但在实际应用中，受限于收发端硬件性能和信号传输环境，DTMB 信号容易受到多径和杂波干扰，使得接收到的信号严重失真，导致雷达目标探测性能下降。为了解决这个问题，需要对参考通道的数据进行重构，以恢复出原发射信号，提高外辐射源雷达目标探测性能。

本文针对 DTMB 外辐射源雷达参考信号在实际应用中受多径干扰和噪声污染导致失真的问题，研究了参考信号重构方法，并实现了基于 CPU-GPU 异构平台的硬件加速。首先，根据 DTMB 信号结构及其时域同步正交频分复用的调制特点，分析了参考信号重构的基本流程，包括信号同步、频偏估计与补偿、信道估计及参考信号提纯等关键步骤。在此基础上，重点研究了改进的时域相关信道估计方法实现，通过避免迭代运算并结合频域处理手段，有效降低了计算复杂度并提高了重构效果。基于这些理论以及实时化的处理需求，设计并实现了基于 CUDA 的 GPU 加速方案，对信号同步、频偏估计与补偿、信道估计及参考信号提纯模块进行并行化优化并设计了 DTMB 参考信号处理系统。其中信道估计实现了原时域相关信道估计方法、频域信道估计算法以及改进的时域相关信道估计方法的 GPU 优化实现。实验结果证明了在本文中实验平台下，该系统能够满足 100 ms 批处理周期下的实时处理要求。

关键词：外辐射源雷达；参考信号重构；GPU 加速；信道估计

ABSTRACT

Passive radar is a bi-/multistatic radar system that exploits third-party non-cooperative illuminators for target detection. It offers several advantages, including strong covertness, enhanced anti-jamming capability, and low cost. China's Digital Terrestrial Multimedia Broadcasting (DTMB) standard, which is based on Time-Domain Synchronous Orthogonal Frequency Division Multiplexing (TDS-OFDM), features wide coverage, strong multipath resistance, stable bandwidth, and an ambiguity function approximating a "thumbtack" shape. These characteristics make DTMB signals suitable illuminators for passive radar systems. However, in practical applications, due to limitations in transmitter and receiver hardware performance as well as complex signal propagation environments, DTMB signals are susceptible to multipath propagation and clutter interference. As a result, the received signal may suffer severe distortion, leading to degraded radar target detection performance. To address this issue, reconstruction of the reference-channel data is required to recover the original transmitted signal and thereby improve the target detection performance of DTMB-based passive radar.

This thesis investigates reference signal reconstruction methods for DTMB-based passive radar, with a focus on reference signal distortion caused by multipath interference and noise contamination in practical scenarios. A hardware acceleration implementation based on a CPU-GPU heterogeneous platform is also developed. First, according to the signal structure of DTMB and the modulation characteristics of TDS-OFDM, the fundamental procedure of reference signal reconstruction is analyzed, including key steps such as signal synchronization, frequency offset estimation and compensation, channel estimation, and channel equalization. On this basis, an improved time-domain correlation-based channel estimation method is studied in detail. By avoiding iterative operations and incorporating frequency-domain processing, the proposed method effectively reduces computational complexity while improving reconstruction performance. Guided by these theoretical analyses and the requirements of real-time processing, a CUDA-based GPU acceleration scheme is designed and implemented. Parallel optimization is carried out for signal synchronization, frequency offset estimation and compensation, channel estimation, and reference signal purification

modules, and a DTMB reference signal processing system is developed. For channel estimation, GPU-optimized implementations of the original time-domain correlation-based channel estimation method, the frequency-domain channel estimation algorithm, and the improved time-domain correlation-based channel estimation method are realized. Experimental results demonstrate that, on the experimental platform used in this thesis, the proposed system satisfies the real-time processing requirement under a 100 ms batch-processing cycle.

Keywords: Passive radar; Reference signal reconstruction; GPU acceleration; Channel estimation

目 录

摘 要	I
ABSTRACT.....	II
1 绪论	1
1.1 研究背景	1
1.2 外辐射源雷达研究现状和发展趋势	1
1.2.1 外辐射源雷达参考信号重构方法研究现状与发展趋势	1
1.2.2 外辐射源雷达参考信号重构的硬件实现现状	2
1.3 本文内容与安排	2
2 DTMB 参考信号重构基础理论	4
2.1 DTMB 信号结构简述.....	4
2.2 DTMB 参考信号重构基本流程.....	5
3 DTMB 外源雷达参考信号重构方法	7
3.1 DTMB 信号降采样方法.....	7
3.2 DTMB 信号同步方法.....	7
3.3 DTMB 信号频偏估计方法.....	9
3.4 信道估计方法	10
3.4.1 信道响应模型	10
3.4.2 时域相关信道估计方法	11
3.4.3 频域信道估计方法	12
3.4.4 最小二乘信道估计方法	13
3.4.5 改进的时域相关信道估计方法	13
3.5 参考信号提纯方法	14
4 参考信号重构在 GPU 硬件平台上的实现.....	17
4.1 GPU 硬件结构及其运算方式	17
4.2 GPU 进行参考信号重构处理流程	19
4.3 GPU 进行信号同步实现方法	20

4.4 GPU 频偏估计加速实现方法	22
4.5 信道估计方法的 GPU 加速实现	24
4.5.1 原时域相关信道估计方法的 GPU 实现	24
4.5.2 频域信道估计方法的 GPU 实现	26
4.5.3 改进的时域相关信道估计方法的 GPU 实现	27
4.5.4 实测性能结果分析	29
4.6 参考信号提纯的 GPU 加速实现	31
4.7 DTMB 参考信号重构的硬件实时化实现仿真.....	33
5 总结与展望	37
5.1 全文总结	37
5.2 后续工作展望	37
参考文献	38
致谢	41

1 绪论

1.1 研究背景

外辐射源雷达即外源雷达，是一种利用非合作第三方辐射源进行目标探测的新型雷达体制^[1]。作为第三方辐射源的 DTMB 信号在国内信号覆盖广泛，功率稳定，且采用 TDS-OFDM 调制方式^[2]，因此基于 DTMB 信号的外辐射源雷达具有无源隐蔽、抗反辐射打击、成本低廉、电磁环境友好等显著优势，在低空监视、边境安防、无人机反制及民用交通监控等领域拥有广阔的应用前景。

然而外辐射源雷达在实际应用中的效果不尽人意。参考通道接收的直达波信号常受多径干扰污染，导致参考信号失真，而且目标通道中的强直达波与杂波会严重掩盖微弱目标回波，因此需要对参考通道的信号进行重构。为了满足外辐射源雷达参考信号的实时化重构的要求，需要用到 GPU 硬件以进行参考信号重构的加速计算。

1.2 外辐射源雷达研究现状和发展趋势

1.2.1 外辐射源雷达参考信号重构方法研究现状与发展趋势

国内外关于外辐射源雷达参考信号重构方法的研究，主要聚焦于从受多径干扰、硬件非理想性及噪声污染的参考通道中恢复高保真直达波信号。国外研究起步较早，多基于 FM、DVB-T 或 LTE 等民用信号^[3-5]，早期采用信道估计^[6]、自适应匹配滤波^[7]或空时自适应处理^[8]等相关的方法，近年来逐渐引入盲源分离和深度学习技术，但这些方法高度依赖信号中的显式导频结构或准静态信道假设，且国外信号（如 DVB-T）采用 CP 作为 OFDM 保护间隔，与我国 DTMB 标准的 PN 序列存在本质差异，难以直接适用。国内武汉大学万显荣教授团队提出了改进的时域相关信道估计算法和基于杂波抑制效果的可信参考信号重构方法^[9]，极大优化了 DTMB 参考信号重构的质量。南昌大学赵志欣教授提出了利用深度神经网络进行 OFDM 参考信号重构的方法并证明了该方法对参考信号重构的有效性^[10]。

当前研究正朝高精度、低复杂度与可工程实现深度融合的方向发展：一方面，利用 DTMB、5G 等信号的先验信息（如帧同步头、PN 序列、导频分布等）设计轻量化的重构方法^[11]，逐步替代传统依赖大量样本或高维矩阵运算的自适应滤波；另一方面，软硬协同设计成为趋势，深度学习技术逐步融入，以实现未知多径

和动态干扰的在线感知与自适应补偿^[12]。此外，面向采样偏差、多径与杂波严重、发射信号不可控等非理想环境下的参考信号重构仍是亟待解决的难题。

1.2.2 外辐射源雷达参考信号重构的硬件实现现状

目前对于参考信号重构的硬件实现方法，大多是利用图形处理器（GPU）来进行加速处理，从而达到实时化处理的目的。文献^[13]中提出了一种基于 GPU 的外辐射源雷达信号处理实时实现方法，用于提高分块最小均方误差算法和 Keystone 算法的并行处理能力。但为了减少硬件平台上 Host（主机端）和 Device（GPU 端）之间的数据传输时间开销，其批处理时长为 1000 ms，这会降低参考信号重构的时效性；文献^[14]则进一步降低了对硬件平台的要求，使用的 CPU、GPU 以及内存等硬件均为消费级产品，每次批处理的数据长度为 128 个信号帧（约为 71ms），能达到每次批处理耗时 54ms 的速度。但是该系统的采样率为 7.56MHz，和 DTMB 信号的码元速率一致，这种等带宽采样的方法会导致 DTMB 信号高频子载波产生混叠，导致信道均衡后的高频子载波符号显著偏离星座点^[15]。

1.3 本文内容与安排

本文核心内容为DTMB外辐射源雷达参考信号重构方法以及其利用GPU硬件平台加速实现方式。具体章节安排如图1.1所示：

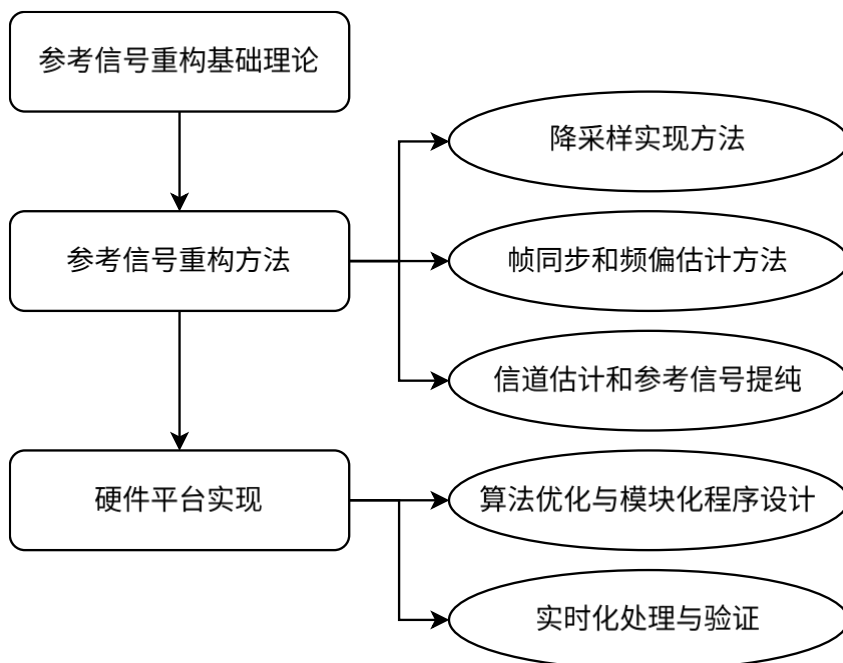


图1.1 本文设计内容和章节安排

第一章为绪论，简单介绍了本文选题的背景与意义，以及外辐射源雷达参考信号重构方法和硬件实现的研究现状与发展趋势，然后给出了本文内容与安排。

第二章为DTMB参考信号重构的基础理论，简述了DTMB信号结构，以及其参考信号重构的基本流程。

第三章为DTMB参考信号重构的方法，给出了实现降采样、信号同步、频偏估计与补偿以及信道估计与参考信号提纯的实现方式，其中信道估计中给出了原时域相关、频域估计、改进的时域相关信道估计三种方法。通过后续的杂波抑制的效果来体现参考信号提纯的效果。

第四章为参考信号重构流程在CPU-GPU异构硬件平台上的实现方式。先简单介绍GPU硬件结构和信号同步、频偏估计与补偿、信道估计和参考信号提纯在CUDA项目上实现的方法，并且提出了改进的时域相关信道估计方法在CUDA项目上的实现，最后通过实验平台来验证利用GPU进行参考信号实时化重构的效果。

第五章总结了全文的设计内容，并给出后续的改进方向。

2 DTMB 参考信号重构基础理论

2.1 DTMB 信号结构简述

DTMB（数字电视地面多媒体广播）是我国自主制定的地面数字电视传输标准，采用TDS-OFDM（时域同步正交频分复用）技术，这种技术同时保证了帧同步的稳定性和数据传输的高效性。DTMB信号的数据帧可以依层次分为日帧、分帧、超帧和信号帧，每一个日帧包含1440个分帧，每个分帧包含480个超帧，每个超帧包含若干个信号帧，其中信号帧由帧头和数据块组成。信号帧有三种帧头模式，分别为PN420模式、PN595模式和PN945模式，这三种模式下，数据帧的结构如图2.1所示。其中PN595模式是单载波模式，帧头平均功率和帧体一致；而PN420和PN945模式都是多载波模式，帧头平均功率是帧体的2倍。

帧头（420个符号，55.6us）	帧体（3780个符号，500us）
帧头（595个符号，78.7us）	帧体（3780个符号，500us）
帧头（945个符号，120us）	帧体（3780个符号，500us）

图2.1 信号帧的三种帧结构

以PN420序列为例，其长度为420个符号，结构上由一个前同步、一个PN255序列以及一个后同步三部分构成，如图2.2所示。其中，PN255序列是长度为255个符号的8阶m序列。前同步和后同步分别是该PN255序列的循环扩展：前同步位于PN255序列之前，长度为82个符号，这个序列与PN255序列的最后82个符号完全相同；后同步位于PN255序列之后，长度为83个符号，与PN255序列的前83个符号完全相同。通过这种结构，整个PN420序列在前后两端分别对PN255序列进行了循环延拓，形成完整的420个符号序列。

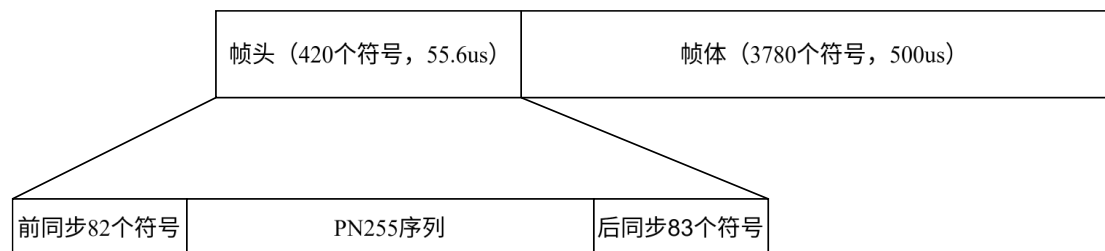


图2.2 PN420序列结构

对于DTMB信号的帧体部分，在频域上表现为3780个相互正交的子载波，每个子载波间隔为2kHz，低频部分的36个子载波为调制系统信息，后面的3744个子载波为数据符号。根据信号质量与环境条件，将数据比特流映射为均匀的nQAM符号流（这里n为星座点数，最先输入的FEC编码比特作为符号的最低有效位），从数据流到符号流支持的映射方式包括64QAM、32QAM、16QAM、4QAM和4QAM-NR，并对各类符号映射引入相应的功率归一化因子，以实现不同调制方式下平均功率的一致性。DTMB信号帧头采用PN序列结构，具备良好的相关特性，因此可同时用于同步、频偏估计和信道估计，这为参考信号重构提供了基础。

2.2 DTMB 参考信号重构基本流程

DTMB参考信号重构的流程本质上是一个“解调-再调制”的过程，需要进行信号同步、频偏估计与补偿、信道估计和参考信号提纯，如图2.3所示。其核心目标是从包含噪声和多径干扰的接收信号中，还原出纯净的原始发射信号，为后续的目标探测提供高质量的参考。

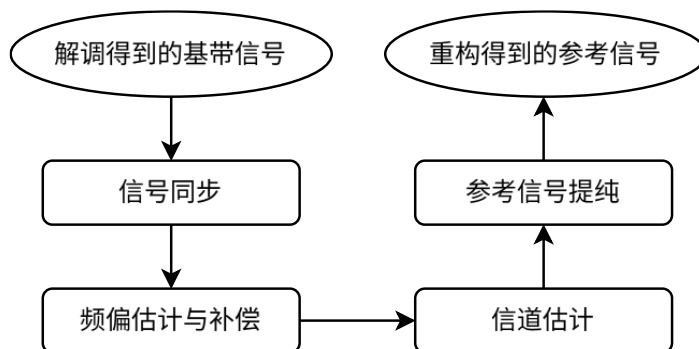


图2.3 参考信号重构基本流程

传统的DBPR（Digital Broadcasting-based Passive Radar，基于数字广播的外辐射源雷达）参考信号重构流程中在信道估计并均衡之后，还要进行解符号交织、解星座映射和解编码得到原始的数据流，然后再根据得到的数据流重新进行信道编码、星座映射和符号交织，再进行OFDM调制并组帧，得到重构的参考信号。这种方式的目的是得到完全无干扰的理想参考信号，然而在实际系统中，发射信号不可避免地受到系统噪声、硬件精度等因素影响，难以得到理想OFDM信号。DTMB信号的系统信息包括符号星座映射模式、LDPC编码码率、交织模式、帧体模式等，利用扩频技术传输。对于DBPR系统，最理想的参考信号就是实际情况下的发射信号。如果发射信号的系统信息已知，则不需要解出原始码流。因此，

这里所使用的参考信号重构流程中没有解交织、解映射和解编码以及重新编码映射交织这些步骤。

利用DTMB信号帧头中伪随机序列优秀的自相关性，系统通过滑动相关运算捕获信号帧的起始位置实现信号同步^[16]。如果收发端的本振信号存在频率误差，或者信号在传输时产生了多普勒效应，接收到的信号就会存在一定的频率偏差。在高斯信道下，要求频偏小于子载波间隔的4%^[17]；在多径信道下，则需小于2%^[18]。DTMB信号的子载波间隔为2kHz，因此如果频偏过大，就需要进行频偏估计和补偿。补偿后允许的频偏误差应小于40Hz。

频偏估计之后，需要进行信道估计和均衡。由于信号在传输过程中经历了多径衰落和畸变，接收到的信号存在码间串扰的情况。在数字信号处理中，这种接收到的信号可以看作发射信号与信道冲激响应的卷积。为了消除信道的影响，需要进行信道估计以得到信道的冲激响应。时域上的卷积相当于频域上的点乘，所以在频域上将接收到的帧体信号除以冲激响应，即可得到原始帧体信号。由于在频域上，TDS-OFDM信号不再像时域那样具有PN序列，因此失去了循环特性，通过该方法还原得到的帧体信号会存在符号间干扰。所以为了消除符号间干扰，还需要将本地PN序列与信道估计结果在时域上进行卷积，以进行循环性重构，具体重构方法会在第三章中介绍。最后结合循环性重构去除保护间隔的符号间干扰，并重新嵌入PN序列帧头，生成完整的纯净参考信号。

3 DTMB 外源雷达参考信号重构方法

3.1 DTMB 信号降采样方法

文献[15]中提到对于带宽为 7.56MHz 的 DTMB 信号，使用等带宽采样会导致其高频子载波部分发生混叠，因而需要使用过采样。在本研究中，使用 10MHz 采样速率对下变频后的基带信号进行采样，然后降采样至 7.56MHz，与 DTMB 信号的符号速率一致。降采样的方法如图 3.1 所示，对于有效频谱为-3.78MHz ~ 3.78MHz 的 DTMB 信号，先将时域信号进行 FFT，截取掉频域-5MHz ~ -3.78MHz 与 3.78MHz ~ 5MHz 的部分（图 3.1 中绿色虚线为截断位置），再进行 IFFT，即可完成降采样。这种方法不会发生频谱混叠，虽然该方法仍会引入一定的频谱泄漏，但对于足够长的数据段，其影响通常可以近似忽略。

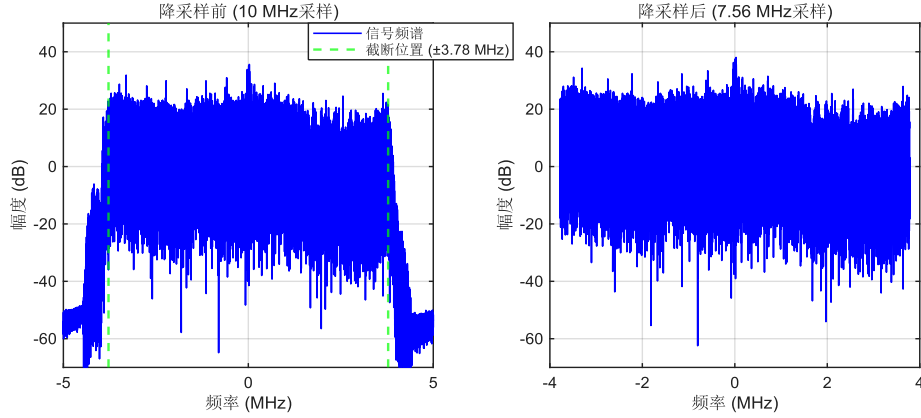


图 3.1 采样得到的信号和降采样后信号频谱

3.2 DTMB 信号同步方法

由于发射站到接收站之间的信号时延与信号帧起始位置未知，所以降采样之后，需要对接收到的信号进行同步，以确定实时接收信号的帧偏移与帧号。若无信号同步，则子载波正交性会被破坏，码间串扰无法避免，这会影响后续的频偏估计和信道估计^[16]。

由于 DTMB 信号的帧头的 PN 序列具有特殊的自相关特性，具有类似随机序列的性质，在一个延时周期 N 内，其自相关函数如式（3.1）所示。

$$R[\tau] = \begin{cases} N, & \tau = 0 \\ -1, & \tau \neq 0 \end{cases}, \tau \in \mathbb{N} \quad (3.1)$$

得益于这种良好的自相关特性，互相关结果会出现明显的主峰，所以可利用本地初始相位 PN 序列 $PN[n]$ 与接收到的 DTMB 信号 $r[n]$ 进行互相关计算。以 PN420 序列为例，其计算方式如表达式（3.2）所示：

$$R[k] = \sum_{n=0}^{N-1} r[k+n] \cdot PN^*[n], \quad N = 420 \quad (3.2)$$

利用这个表达式，实测互相关结果如图 3.2 所示。

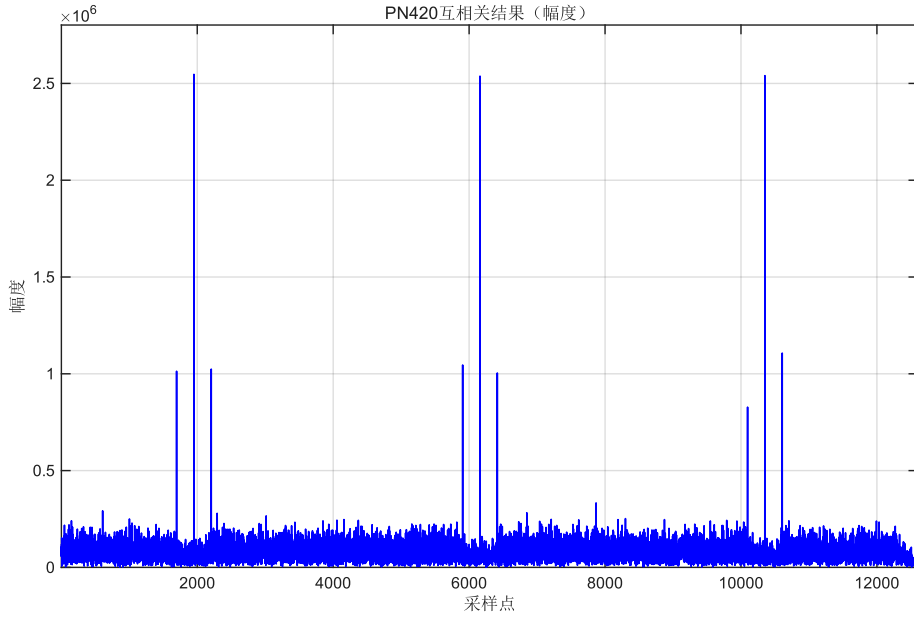


图 3.2 PN420 与接收信号互相关计算结果

由互相关结果可以看出，每相隔 4200 个点左右，会有一个很强的主相关峰。这个主相关峰的左右两边紧挨着次峰，这是接收数据帧头以及本地 PN420 序列中的循环前缀和循环后缀的互相关峰导致的。利用两个主相关峰之间的间隔，可以确定帧偏移以及该信号帧的帧号。下文开始介绍确定帧偏移和帧号的原理以及方法。

对于 DTMB 信号，相邻信号帧的 PN 序列相位偏移是确定的，利用 PN 序列的相位偏移，可以推算出这两个相邻信号帧的帧号。以 PN420 模式为例，相邻信号帧 PN 序列相位偏移可以表示为 Δp ， Δxp 表示互相关结果中相邻强相关峰的间隔， len 表示信号帧长度。则 Δp 的表达式如下：

$$\Delta p = \Delta xp - len \quad (3.3)$$

在 PN420 模式下，信号帧长度为 4200。第一个信号帧的帧号 $frame$ 与相位偏移 Δp 关系可通过表达式 (3.4) 得到：

$$frame = \text{mod}(225 \cdot \text{mod}(|\Delta p|, 2) + (-1)^{\text{mod}(|\Delta p|, 2)} \cdot \Delta p, 225) \quad (3.4)$$

在计算得出帧号以后，可以根据第一个强相关峰的位置进而得到帧偏移。帧偏移 $bias$ 可以通过表达式 (3.5) 得到：

$$bias = \text{mod}(n_0 - bias_{\text{xcorr}} - 1, 4200) \quad (3.5)$$

其中 $bias_{\text{xcorr}}$ 与相位偏移 Δp 的关系可由表达式 (3.6) 得到：

$$bias_{\text{xcorr}} = \begin{cases} \lfloor |\Delta p|/2 \rfloor, & \Delta p < 0 \\ -\lfloor |\Delta p + 1|/2 \rfloor, & \Delta p \geq 0 \end{cases} \quad (3.6)$$

这种方法只需要利用到接收到的信号和本地初始相位的 PN420 序列即可得到帧偏移结果。如果能根据得到的帧号选择相应相位的本地 PN420 序列，则可以直接将接收到的信号和这个 PN420 序列进行互相关计算，得到的第一个强相关峰对应的位置就是帧偏移的位置。从帧偏移位置开始为第一个数据，则接收到的时域数据的前 420 个点就是该帧号对应相位的 PN420 序列。信号同步结果可以为后续的频偏估计提供准确的同步基础。

3.3 DTMB 信号频偏估计方法

在完成信号同步之后，还需要对接收信号中的载波频偏进行估计与补偿。载波频偏的产生主要来源于收发两端本振频率的不一致以及信号传播过程中的多普勒频移。假设接收信号存在载波频偏 Δf ，本地 PN 序列为 $PN[n]$ ，理想情况下的符号周期为 T ，如果忽略噪声信号和其他影响因素，则接收到的 PN 序列信号 $r[n]$ 可表示为：

$$r[n] = PN[n] \cdot e^{j(2\pi\Delta f nT)} \quad (3.7)$$

频偏估计有多种方法，但频偏估计的效果只有精度和估计范围的区别。下面介绍一种利用 PN 序列内部相关估计的方法。将接收到的信号 $r[n]$ 和对应相位的本地 PN 序列进行共轭相乘，得到表达式 (3.8)：

$$z[n] = r[n] \cdot PN^*[n] = |PN[n]|^2 \cdot e^{j(2\pi\Delta f nT)} \quad (3.8)$$

对得到的 $z[n]$ 进行自相关，则有：

$$R[l] = \sum_{n=l}^{N-1} z[n-l] \cdot z[n] = (N-l)|PN[n]|^4 \cdot e^{-j(2\pi\Delta f nT)} \quad (3.9)$$

其中 l 为 PN 序列的延迟长度。由于 $(N-l)|PN[n]|^4$ 为实数，则可以直接得到频偏 Δf 为：

$$\Delta f = -\frac{\angle(R[l])}{2\pi Tl} \quad (3.10)$$

由于该方法仅利用本帧长度为 255 的 PN 序列进行估计，因此其可估计频偏 Δf 的范围很大，但估计精度很低。为了提高频偏估计的精度，可以使用多帧的 PN 序列的关系来进行频偏估计，这样对应的延迟长度 l 可以变得很大。当间隔的符号数为两帧，即 8400 左右时，最大范围为：

$$|\Delta f| < \frac{\pi}{2\pi * 132.3 \text{ ns} * 8400} = 450 \text{ Hz} \quad (3.11)$$

此时的频偏估计的精度可以非常高，满足后续进行信道估计和参考信号提纯等需求。得到信号的频偏之后，可以对接收到的数据进行频偏补偿，从而能够恢复出接收信号的各子载波的正交性，确保了得到的星座图不会出现旋转的情况。

3.4 信道估计方法

3.4.1 信道响应模型

信道估计是 DBPR 参考信号重构中的最关键的步骤之一，信道估计的结果会直接影响到后续的参考信号提纯的质量。信道估计可以看作是得到信道的冲激响应，然后利用得到的冲激响应去处理接收到的信号，从而恢复出纯净的无多径的参考信号。

对于传输信道，可以看作是直达路径和多个不同时延的多径响应的线性叠加。由于多径信道的时延相对于 PN 序列的时间长度较短，所以一般情况下多径信道的时延不会超过 PN 序列的循环前缀部分。在数字信号域中，可以假设有直达路径和 $N-1$ 个多径信道，如果这些信道的响应幅度分别为 a_n ，则在时域，信道响应可以表示为：

$$h[n] = \sum_{k=0}^{N-1} a_n \delta[n-k] \quad (3.12)$$

假设 $s[n]$ 是无频偏的发射信号， $w[n]$ 是加性高斯白噪声，则接收到的信号 $r[n]$ 可以表示为：

$$r[n] = \sum_{k=0}^{N-1} a_n s[n-k] + w[n] \quad (3.13)$$

3.4.2 时域相关信道估计方法

由于一般多径信道的时延不会超过 PN 序列的循环前缀的部分，所以接收到的信号中，去掉循环前缀部分的 PN 序列是没有经过前一信号帧多径信号污染的序列，因此可以用来进行信道估计^[19]。以 PN420 序列为例，将去掉循环前缀的 255 点 PN 序列设为 $PN_1[n]$ ，将这个序列与接收信号中对应相位的去掉循环前缀的 PN 序列部分进行互相关计算，得到表达式 (3.14)：

$$R_t[n] = \sum_{k=0}^{M-1} r[n+k]PN_1[n], \quad n \in [0, N-1], n \in \mathbb{N} \quad (3.14)$$

其中 N 为信道长度，M 为 $PN_1[n]$ 序列的长度，即 255。将表达式 (3.13) 代入到表达式 (3.14) 中，则得到表达式 (3.15)：

$$R_t[n] = \sum_{k=0}^{M-1} PN_1[n] \left(\sum_{i=0}^{N-1} a_n s[n+k-i] + w[n+k] \right) \quad (3.15)$$

利用 PN 序列的循环拓展关系，可以得到表达式 (3.16)：

$$s[n+k-i] = PN_1[\text{mod}(n+k-i, M)] \quad (3.16)$$

然后将表达式 (3.16) 带入表达式 (3.15)，从而得到：

$$\begin{aligned} R_t[n] &= \sum_{k=0}^{M-1} \sum_{i=0}^{N-1} a_n PN_1[n] PN_1[\text{mod}(n+k-i, M)] + \sum_{k=0}^{M-1} PN_1[k] w[n+k] \\ &= \sum_{i=0}^{N-1} a_n R_p[n-i] + \sum_{k=0}^{M-1} PN_1[k] w[n+k] \end{aligned} \quad (3.17)$$

这样得到的 $R_p[n]$ 是对应相位的本地 PN_1 序列的自相关函数。而 PN 序列与噪声几乎不相关，所以 $\sum_{k=0}^{M-1} PN_1[k] w[n+k]$ 这一项也可以看作是噪声。如果 PN_1 序列是理想的冲激函数，那么 $R_t[n]$ 就是时域信道响应。然而 PN_1 序列并非理想的冲激函数，对于复数形式的 PN_1 序列，类似于表达式 (3.1)，其自相关结果为：

$$R_{PN_1}[n] = \begin{cases} M, & n = 0 \\ -1, & n \neq 0 \end{cases}, \quad n \in \mathbb{N} \quad (3.18)$$

从这个表达式可以看出， PN_1 序列的自相关函数结果在非零处并不为 0。也就是说，以这种方式得到的时域信道响应结果实际上是此处的多径和其他所有多径的 $-1/M$ 倍幅值的叠加的结果。为了抑制其他位置的多径对此处多径的影响，需要额外进行干扰消除。有一种干扰消除的思路是，根据幅度较大的多径收到其他

幅度较小的多径信号的影响较小的性质，设计迭代消除法。迭代消除法的具体实现步骤为：先找到信道响应的主径，对这个主径进行干扰消除以补偿主径，主径补偿后继续寻找幅度第二强的副径继续进行干扰消除，如表达式（3.19）所示：

$$R_{tc}^{(k)}[n] = \begin{cases} \frac{R_{tc}^{(k-1)}[n]}{M} + \frac{a_k}{M^2}, & n = n_k \\ \frac{R_{tc}^{(k-1)}[n]}{M}, & n \neq n_k \end{cases} \quad (3.19)$$

其中 k 为迭代次数， a_k 为第 k 强多径的幅度， n_k 为这个多径相应的位置。最终经过 K 次迭代消除之后，得到补偿后的时域信道估计结果 $\hat{h}_t[n]$ 如(3.20)所示：

$$\hat{h}_t[n] = R_{tc}^{(K)}[n] \quad (3.20)$$

3.4.3 频域信道估计方法

由于时域信道估计方法中需要进行迭代相消，迭代算法十分不利于并行计算的实现，这会严重影响运行速度。而频域信道估计方法直接在频域上进行，然后直接设置门限进行干扰信道消除，因此不需要进行迭代相消，这可以极大地提升运行速度^[20]。对于接收到的信号如表达式（3.13）所示，对于接收到的帧头的 PN 信号后 255 点序列 PN_1 可以表示为：

$$r_{PN1}[n] = \sum_{l=0}^{N-1} a_n PN_1[n-l] + w[n] \quad (3.21)$$

对接收到的 PN_1 序列进行离散傅里叶变换，则有：

$$\begin{aligned} R_{PN1}[k] &= FFT\left(\sum_{l=0}^{N-1} a_n PN_1[n-l] + w[n]\right) \\ &= FFT\left(\sum_{l=0}^{N-1} a_n PN_1[n-l]\right) + W[k] \\ &= \hat{R}_{PN1}[k] + W[k] \end{aligned} \quad (3.22)$$

其中 $\hat{R}_{PN1}[k]$ 为只含多径的 PN_1 序列的频谱， $W[k]$ 是加性高斯白噪声的频谱。对本地相应相位的 PN_1 序列 $s_{PN1}[n]$ 进行离散傅里叶变换，得到其频谱表达式如（3.23）所示：

$$S_{PN1}[k] = FFT(s_{PN1}[n]) \quad (3.23)$$

由于在一个信号帧内的信道响应的变化可以忽略不计，所以可以直接得到信道估计结果 $\hat{H}[K]$:

$$\hat{H}[k] = \frac{R_{PN1}[k]}{S_{PN1}[k]} \quad (3.24)$$

将这个频域信道估计结果由频域转化为时域，则得到:

$$\hat{h}[n] = \text{IFFT} \left(\frac{R_{PN1}[k]}{S_{PN1}[k]} \right) \quad (3.25)$$

3.4.4 最小二乘信道估计方法

由于多径效应一般不会延伸至 PN 序列的循环前缀后面的部分，所以去掉循环前缀的 PN 序列不会受到前一帧的帧体信号干扰。因此可以利用 PN 序列去掉循环前缀的部分来进行最小二乘信道估计^[21]。接收到的帧头中后 255 点序列 PN_1 的表达式可以表示为:

$$r = Ph + w \quad (3.26)$$

其中 P 为本地 PN 序列构成的矩阵，矩阵内的 PN 为初始零相位的本地 PN 序列。 $h[n]$ 为信道响应， $w[n]$ 为加性高斯白噪声。其中矩阵 P 、 h 、 r 和 w 分别表示为:

$$P = \begin{bmatrix} PN[L-M] & \cdots & PN[L-M-N+1] \\ \vdots & \ddots & \vdots \\ PN[L-1] & \cdots & PN[L-N] \end{bmatrix} \quad (3.27)$$

$$h = [h[0] \ h[1] \ \dots \ h[N-1]]^T \quad (3.28)$$

$$w = [w[0] \ w[1] \ \dots \ w[M-1]]^T \quad (3.29)$$

$$r = [h[L-M] \ h[L-M+1] \ \dots \ h[L-1]]^T \quad (3.30)$$

其中 L 为 PN 序列的长度， M 为 PN 序列去掉循环前缀的长度， N 为信道长度。 L 、 M 、 N 之间的关系要满足 $L \leq M + N - 1$ 。根据最小二乘法的计算方式，得到的信道估计结果 $\hat{h}$ 为:

$$\hat{h} = (P^H P)^{-1} P^H r \quad (3.31)$$

3.4.5 改进的时域相关信道估计方法

由于原时域信道估计算法需要进行迭代消除，最小二乘信道估计方法需要进行矩阵求逆，这两种方法的计算复杂度都很高。而改进的时域相关信道估计方法通过求解线性方程，不需要进行迭代消除，可以直接估计得到信道响应^[15]。根据式（3.18）中 PN 序列的自相关特性，式(3.17)可以化简为:

$$R_t[n] = (M + 1)h[n] - \sum_{k=0}^{N-1} h[k] \quad (3.32)$$

上式两边经过求和，得到式（3.33）：

$$\sum_{n=0}^{N-1} R_t[n] = (M + 1 - N) \sum_{k=0}^{N-1} h[k] \quad (3.33)$$

将表达式（3.33）再代入式（3.32）计算，可以得到改进的时域相关信道估计结果 $\hat{h}_t[n]$ 如式（3.34）所示^[15]：

$$\hat{h}_t[n] = \frac{(M - N + 1)R_t[n] + \sum_{k=0}^{N-1} R_t[k]}{(M - N + 1)(M + 1)} \quad (3.34)$$

这种改进的时域相关信道估计方法通过直接求解线性方程，避开了 GPU 并行计算中不擅长的循环依赖迭代计算，可以显著的降低时间复杂度。时域相关信道估计、频域信道估计以及改进的时域相关信道估计方法的运行性能将在第四章的 GPU 信道估计方法实现中展示。

3.5 参考信号提纯方法

DTMB 外辐射源雷达系统中，接收的参考信号包含两部分有用信息：一是帧头的 PN 序列，二是承载数据的帧体部分。由于多径的影响，本数据帧的帧头的前面部分被上一帧的帧体的后面部分污染，本数据帧的帧体的前面部分也会被上一数据帧的帧头后面部分污染，这会引入符号间干扰^[22]，如图 3.3 所示。

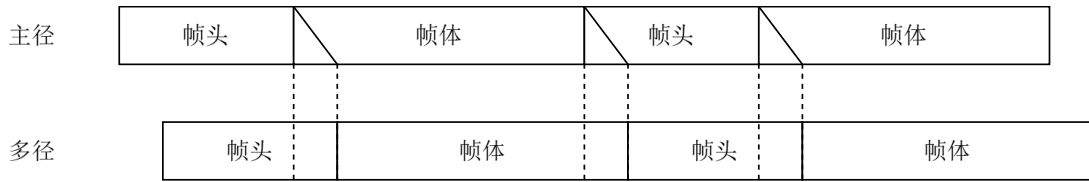


图 3.3 符号间干扰影响示意图

由此可以看出，帧体数据不仅会受到多径部分的帧体数据的影响，还会受到帧头 PN 序列的影响，因此需要进行循环性重构^[23]，以消除这样的符号间干扰。循环性重构的核心思路是，利用相邻帧 PN 序列的已知信息，估计并消除保护间隔处的符号间干扰。具体步骤为，对每一帧的数据，计算该数据帧对应的本地 PN 序列与信道响应结果的卷积 g_1 和下一帧对应的 PN 序列与信道响应结果的卷积 g_2 。假设该数据帧的帧号为 k ，该帧对应的 PN 序列为 PN_k ，下一数据帧的帧号为 l ，卷积结果如表达式（3.35）所示：

$$\begin{aligned} g_1 &= PN_k \otimes \hat{h} \\ g_2 &= PN_l \otimes \hat{h} \end{aligned} \quad (3.35)$$

对于 PN420 模式，卷积结果的长度为 $L + N - 1 = 674$ 。得到卷积结果后，再对帧体受到 PN 序列干扰的部分进行消除。对于帧体数据 r_{bk} 中受到干扰的部分，也就是前 N 个接收到的帧体数据，其循环性重构结果如表达式（3.36）所示：

$$r_{bk}[n] = r_{pk}[n + 420] + r_{pl}[n] - g_1[n + 420] - g_2[n] \quad (3.36)$$

其中 $n \in \mathbb{N}$, $n < N$, r_{pk} 为接收到的该数据帧所有数据, r_{pl} 为接收到的下一数据帧的所有数据。对于 r_{bk} 后面没有受到 PN 序列和信道响应卷积结果干扰的数据则保持不变。循环性重构结束之后，则开始进行信道均衡。由于时域中信道响应和数据帧的卷积在频域上相当于信道响应和数据帧的相乘，所以先将信道估计结果 $\hat{h}$ 和循环性重构之后得到的帧体数据 r'_b 分别进行离散傅里叶变换，得到：

$$H[k] = FFT(\hat{h}) \quad (3.37)$$

$$R_b[k] = FFT(r'_b) \quad (3.38)$$

则信号帧的帧体在时域上的信道均衡结果 $\hat{r}_b$ 为：

$$\hat{r}_b = IFFT\left(\frac{H[k]}{R_b[k]}\right) \quad (3.39)$$

由于本地 PN 序列和接收信号帧的帧号已知，所以信号帧的帧头可以直接使用对应相位的本地 PN 序列填充，使得重构后得到的参考信号更为纯净。最终提纯得到的参考信号 $\hat{r}$ 如表达式（3.40）所示。对于 PN420 模式， L 为 420， Len 是数据帧的长度，为 4200。

$$\hat{r} = \begin{cases} PN[n], & n = 0, 1, \dots, L - 1 \\ \hat{r}_b, & n = L, L + 1, \dots, Len - 1 \end{cases} \quad (3.40)$$

利用提纯得到的参考信号，可以结合监测通道 1 的信号进行空域杂波抑制，参考信号提纯前后通过杂波抑制得到的距离多普勒谱结果如图（3.4）和图（3.5）所示。使用数据为 2020 年 9 月洛阳参考信号多径实验中参考方向指向发射站的数据，提纯过程中所利用的信道估计方法是时域相关信道估计。

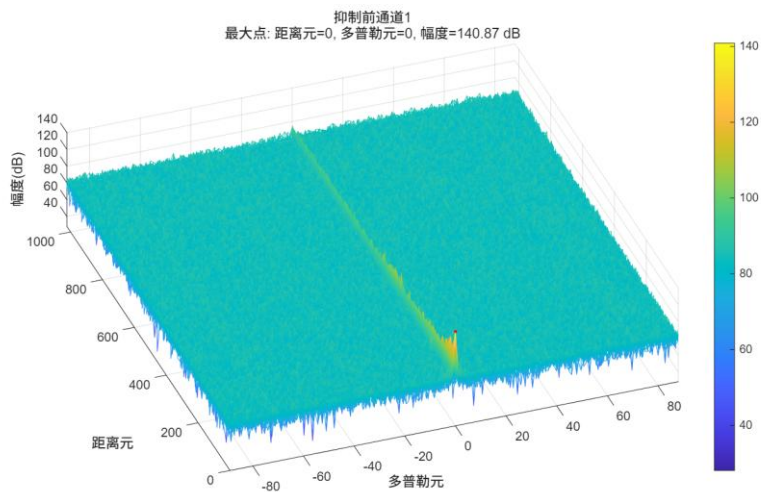


图 3.4 利用提纯前信号得到的 RD 谱结果

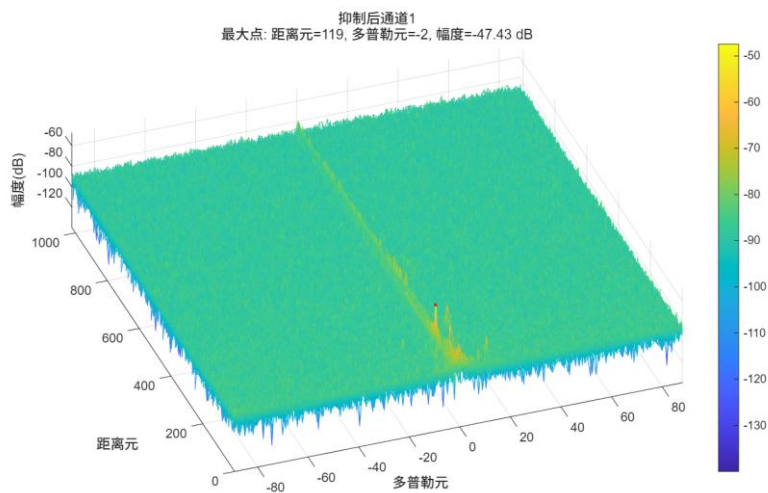


图 3.5 利用提纯后信号得到的 RD 谱结果

通过图（3.4）和图（3.5）可以看出，参考信号提纯前得到的 RD 谱的峰值在距离维和多普勒维的位置都为 0，这是因为参考通道和监测通道都受到了几乎是相同特性的多径信道的影响；参考信号提纯后得到的 RD 谱的峰值在距离和多普勒维上的位置均不为 0，可以清晰的看到主要探测目标的距离和速度，这表明所提出的参考信号重构方法能够有效改善后续杂波抑制效果，并提升目标在距离-多普勒谱中的可辨识度。

4 参考信号重构在 GPU 硬件平台上的实现

4.1 GPU 硬件结构及其运算方式

为了加速对 DTMB 参考信号重构的处理，可以利用 GPU 硬件平台来加速实现参考信号重构流程。计算机的计算过程在 CPU 和 GPU 上进行，如图（4.1）所示。这样的 CPU-GPU 异构计算架构中，CPU 作为主机（Host）端，GPU 作为设备（Device）端，二者通过 PCI-E 总线进行通信。

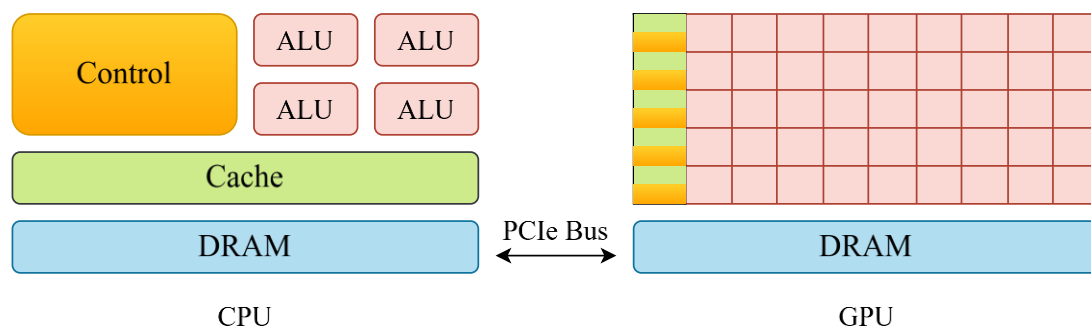


图 4.1 CPU 和 GPU 的硬件结构

由图（4.1）可以看出，CPU 硬件结构中包含控制单元、若干个算术逻辑单元（ALU）、高速缓存单元和动态随机存储单元（DRAM）。而 GPU 中除了 DRAM 外，只有十分简单的控制和逻辑单元，剩下的大半面积是数量众多的计算单元和极长的流水线。以本项目使用的硬件平台为例，CPU 为 Intel® Core™ i9-13900HX，使用 Raptor Lake 架构，拥有 8 个性能核心与 16 个能效核心，L3 高速缓存大小为 36MB。GPU 为 NVIDIA GeForce RTX 4060 Laptop，属于 Ada Lovelace 架构的简化版芯片，拥有 3072 个 CUDA 核心和 24 个流式多处理器（SM）、32MB 的 L2 缓存，其显存为 8GB GDDR6。完整的 Ada Lovelace 架构简易示意图如图（4.2）所示。可以看到多个图形处理集群（GPC）共享 L2 高速缓存，GPC 是 NVIDIA GPU 架构中的顶层硬件模块，也是构成 GPU 核心计算资源的基本单元。每个 GPC 是一个完全独立的处理单元，包含完整的计算、纹理和输出资源。除此之外，该架构还有线程调度引擎、光流加速器（OFA）、硬件视频解码器（NVENC）及数个内存控制器。

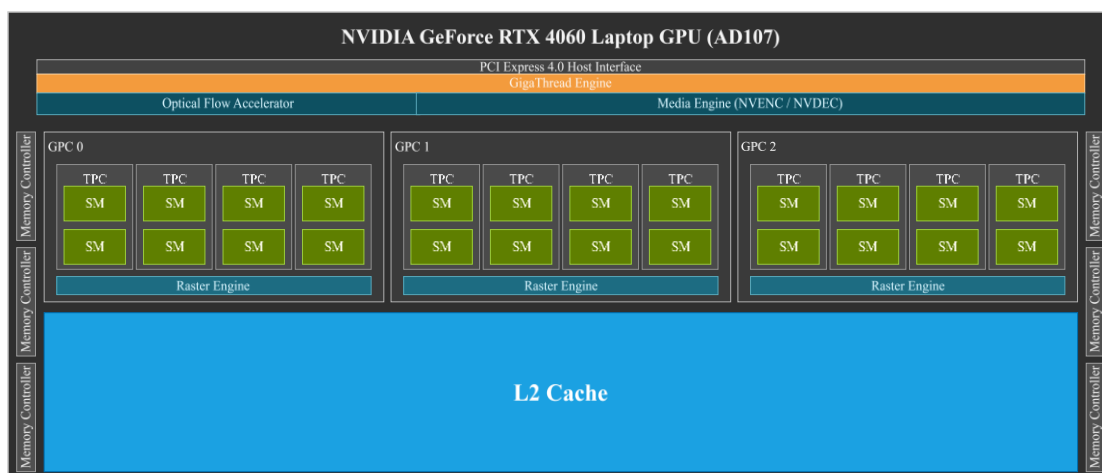


图 4.2 Ada Lovelace(RTX 4060 Laptop)架构简易示意图

对于中低端定位的 NVIDIA GeForce RTX 4060 Laptop GPU, 其拥有 3 个 GPC, 每个 GPC 中包含 8 个 SM 单元。SM 单元的结构如图（4.3）所示。

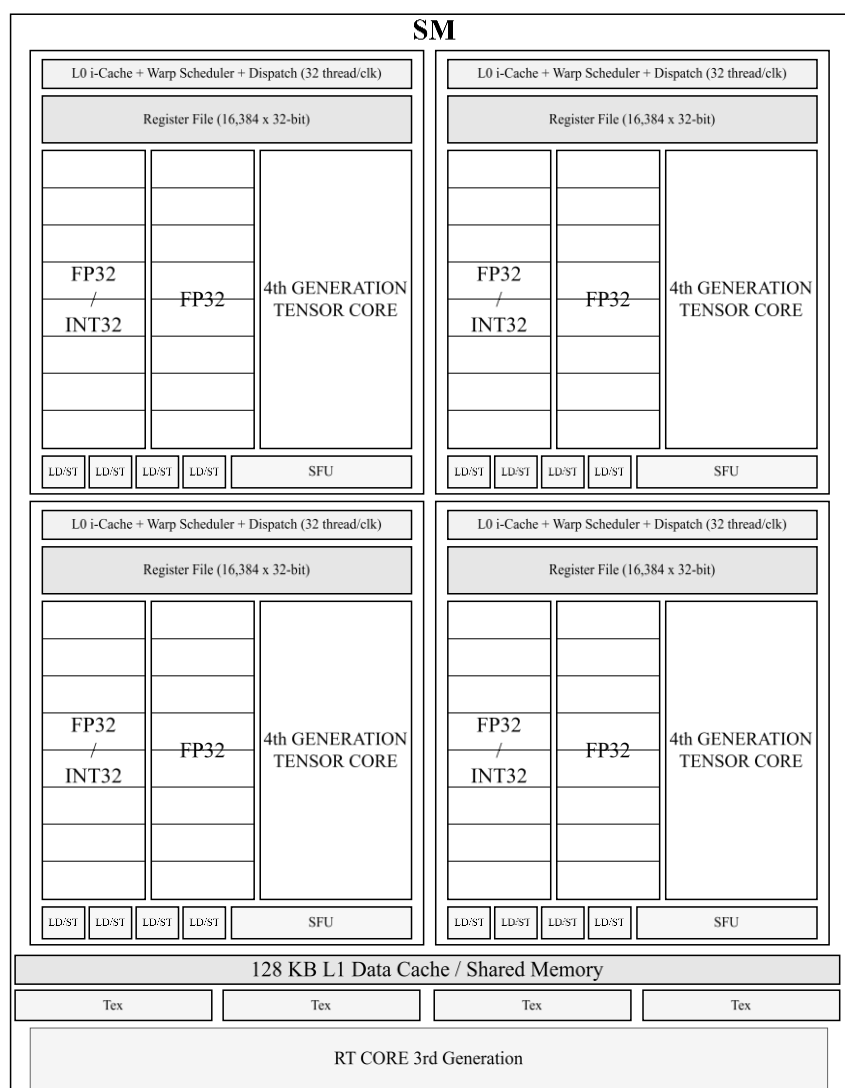


图 4.3 SM 单元结构示意图

可以看出，每个 SM 单元内部都被独立划分为 4 个独立的分区，每个分区包含 32 个 CUDA 核心、调度器、SFU（特殊执行单元）、光追核心（RT Core）以及 1 个张量核心（Tensor Core）。其中 SFU 可以用来执行超越函数（如 $\sin$ 、 $\cos$ 、 $\log$ 、平方根倒数等），它与同一分区内的 CUDA 核心共享部分硬件资源。其中本项目使用的 RTX 4060 Laptop 的计算能力为 8.9，这个计算能力可以看作是 NVIDIA GPU 系列的架构版本号，这使得其支持大部分现代 GPU 计算特性。

4.2 GPU 进行参考信号重构处理流程

此项目的最终目的是实现 DTMB 外辐射源雷达参考信号重构这一过程在 CPU-GPU 硬件平台上的加速化甚至实时化实现。为了实现这一目的，需要尽可能将主要计算过程在 GPU 上进行，并且减少 CPU 端(Host)与 GPU 端(Device)的数据通信量，以加快运算速度、降低在数据传输上的延迟。如果有多个监测通道，则后续可以进行波束形成。信号处理主要过程如图（4.4）所示。

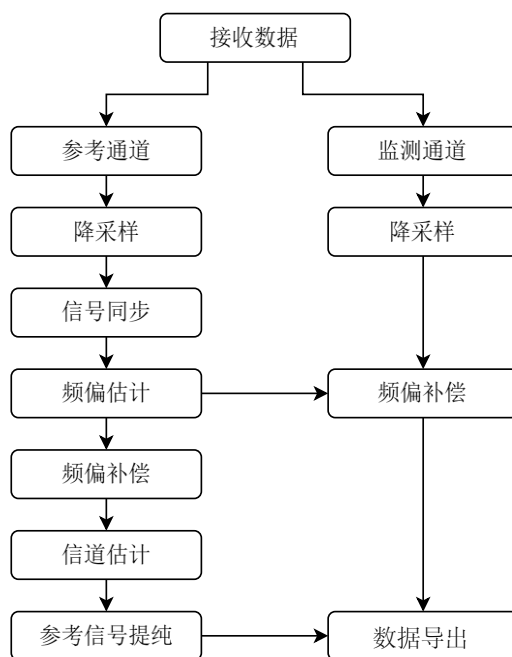


图 4.4 信号处理流程

整个 DTMB 参考信号重构系统的输入的数据为三路采样率为 10MHz、数据类型为 double 型的正交信号，其中一路是参考通道的数据，另外两路是监测通道的数据。接收采用批处理的方式，每次批处理 100ms 长度，也就是 48MB 大小的数据量。对于采用 PN420 模式的 DTMB 信号，每 100ms 长度的数据是 180 数据帧。对于参考通道的信号，先进行降采样，得到采样速率和符号速率一致的复数基带信号。然后利用得到的信号进行同步得到帧偏移和当前帧号，结合信号同步

的结果进行频偏估计，然后利用频偏估计的结果进行频偏补偿。之后，利用频偏补偿的信号进行信道估计，最后利用信道估计的结果进行循环性重构、均衡并组帧，得到重构之后的信号。而另外两路的监测通道数据则进行降采样，然后利用频偏估计的结果进行频偏补偿，直接得到处理后的监测通道数据。系统同时输出这两路的数据，可以使得其进行后续的杂波抑制，以得到最后的距离多普勒谱，进行目标的探测。

4.3 GPU 进行信号同步实现方法

DTMB 系统采用的同步算法基于 PN 序列的互相关特性。在第三章信号同步方法提到过，PN 序列具有良好的自相关特性，当接收信号中的 PN 序列与本地生成的 PN 序列对齐时，互相关值达到最大。通过搜索互相关结果的峰值位置，即可确定某一信号帧的起始边界。以 PN420 模式为例，DTMB 信号每帧包含 4200 个采样点，其中包含特定的 PN 序列结构。同步算法通过检测两个相邻帧的 PN 序列位置，计算出信号偏移和帧号。传统的 CPU 实现方式受限于计算核心的数量与规模，难以满足实时处理大规模数据的需求。DTMB 信号具有帧结构固定、PN 序列已知等特点，这为并行化处理提供了良好的条件。为减少 GPU 内存分配和释放的开销，系统采用缓存机制预分配设备内存，PN420 参考序列只需传输一次到设备端，后续处理直接使用缓存数据，大幅减少了数据传输开销。GPU 实现充分利用了 CUDA 架构的并行特性^[24]，将不同滞后位置的互相关结果运算分配给各线程，以进行加速计算^[25]。

GPU 并行实现采用了一维网格跨步映射（Grid-Stride Loop）策略。根据硬件资源特性，CUDA 网格被设计为 64 个线程块，每个线程块包含 256 个线程，总计提供 16384 个并发线程，如图（4.5）所示。由于 DTMB 帧同步的最大延迟为 12600 点，小于系统并发的总线程数，因此互相关计算的负载几乎被均匀地分配给各个线程：每个线程被独立分配到一个具体的滞后位置（Lag），并在该线程内部顺序完成重叠长度内的互相关累加操作。这种分配方式使得所有滞后位置的相关计算能够完全并行执行。由于每个 SM 单元的最大并发线程数为 1536，如果每个线程块的线程数设置为 256，则一个 SM 可以同时驻留 6 个 Block（ $256 \times 6 = 1536$ ）。这样 SM 单元的并发线程数就达到了其理论上限，其计算资源理论上可以得到充分利用。

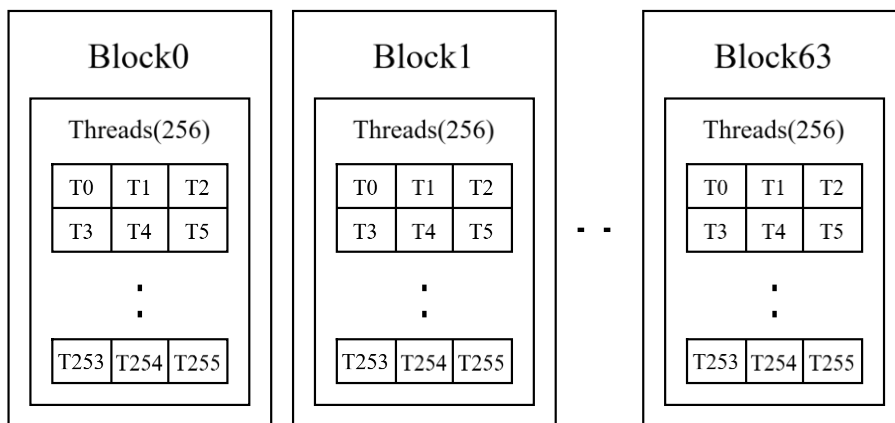


图 4.5 GPU 线程块与线程

此外，DTMB 系统的本地 PN 序列（PN420）在基带可等效为实数双极性序列，仅有正负两个取值。内核充分利用这一特点对数据与本地 PN420 序列的乘积运算进行优化，在互相关遍历中，当本地 PN 值为正时，线程直接执行复数累加；当 PN 值为负时，则执行复数累减。这种优化将高算力开销的复数乘累加运算等效转化为高效的加减法运算，大幅提升了 CUDA 程序的运算速度。

利用 GPU 加速计算信号同步和在 Matlab 上处理的结果如表 4.1 所示。其中两个项目都是对采集到的相同的信号进行 10 次同步，Index0 和 Index1 分别表示互相关结果中的最强峰值对应的索引，表中计算结果都是第一次批处理得到的结果，运行时间是进行 10 次批处理所消耗的平均时间。Matlab 项目使用 CPU 的 24 核心 32 个线程进行信号同步计算，CUDA 项目中利用 NVIDIA Nsight System 性能分析工具进行分析，使用 NVTX 标签从调用信号同步核函数开始计时，到返回结果结束计时。

表 4.1 GPU 和 Matlab 进行信号同步的结果

运行项目	Index0	Index1	信号偏移	帧号	平均运行时间
CUDA	1937	6173	1954	36	241.6 μ s
Matlab	1938	6174	1955	36	5.3ms

在 Matlab 中的索引从 1 开始，而 CUDA 项目中的索引从 0 开始，所以 CUDA 项目中 Index 和信号偏移的结果都比 Matlab 项目中的结果小 1，这是符合预期的结果。由表 4.1 中结果看出，Matlab 和 CUDA 项目运行结果中信号偏移和帧号结果一致，可以说明二者信号同步得到的结果是正确的。此外，可以看出 CUDA 项目的运行时间显著短于 Matlab 项目的运行时间，前者运行速度大约是后者的 22 倍，由此可以说明利用 GPU 进行信号同步加速计算的有效性。

4.4 GPU 频偏估计加速实现方法

频偏估计已经在本文 3.3 节中提到，其结果会直接影响到后续信道估计和参考信号提纯的效果，这里详细介绍利用 GPU 实现频偏估计的方法。本项目中所采集的信号频偏范围往往小于 450Hz，所以可以使用精频偏估计方法。频偏估计模块中经过信号同步的参考通道数据从主机端(Host)发送到 GPU(Device 端)上，GPU 启动并行 kernel，同时计算所有数据帧的频偏估计值，然后将计算得到的各信号帧的频偏结果传回主机端。为了实现信号处理的实时性，需要避开在 GPU 上频繁的进行分配和销毁内存的操作，因此在频偏估计模块启动时，就根据预设的最大帧数和最大数据长度(max_len)，一次性预分配足够的全局显存资源，包括输入数据、相关操作缓存和输出频偏结果所用到的显存。本地 PN420 序列经过极性转换和归一化的预处理后，就长期保存在 GPU 全局内存中，避免了频繁重复地传输已知 PN 序列，减少数据传输消耗的时间。

为了最大化 GPU 的流式多处理器(SM)的利用率，本文采用了一维的 Grid 和 Block 划分策略^[26]。在自定义的 CUDA 核函数 compute_correlation_kernel 中，将待处理的信号帧数映射为 Grid 中的 Block 数量(blockIdx.x)，即每个 Block 独立负责一对帧的互相关计算与相位提取。综合考虑 GPU 占用率、线程调度开销和共享内存使用情况，本文项目设计为每个线程块固定分配 256 个线程(threadIdx.x)，使得 420 个采样点的相关乘积累加任务能够均匀分布到各个线程中，有效隐藏了指令延迟。每个线程遍历 PN 序列的有效长度，完成以下操作：根据索引计算当前 PN 点在数据中的位置；读取第 m 帧和第 m+2 帧中对应位置的数据；查询 PN 序列值；计算共轭相关 $z = y \cdot \text{conj}[PN]$ ；将相关结果累加至共享内存。

由于同一线程块内的线程需要对其负责的乘积结果进行求和，如果直接写入全局内存(Global Memory)将引发严重的写冲突和性能瓶颈。本设计引入了基于共享内存的并行归约(Parallel Reduction)策略。代码设计思想如下所示：首先将各线程在寄存器中完成复数乘积计算，并将结果写入共享内存数组 sh_sum 和 sh_sum_short 中（分别对应全长 420 点和截断长度 156 点的相关求和）。然后通过 __syncthreads() 进行块内线程同步，再采用二分法步长(128, 64, 32...)进行规约累加，将 $O(N^2)$ 的时间复杂度降至 $O(N \log(N))$ 。最终由主线程提取归约结果

的复数相位，并根据 DTMB 系统参数及同步帧模式计算出实际的频率偏移值。其中归约示意如图（4.6）所示（以内存数组 sh_sum 为例）。

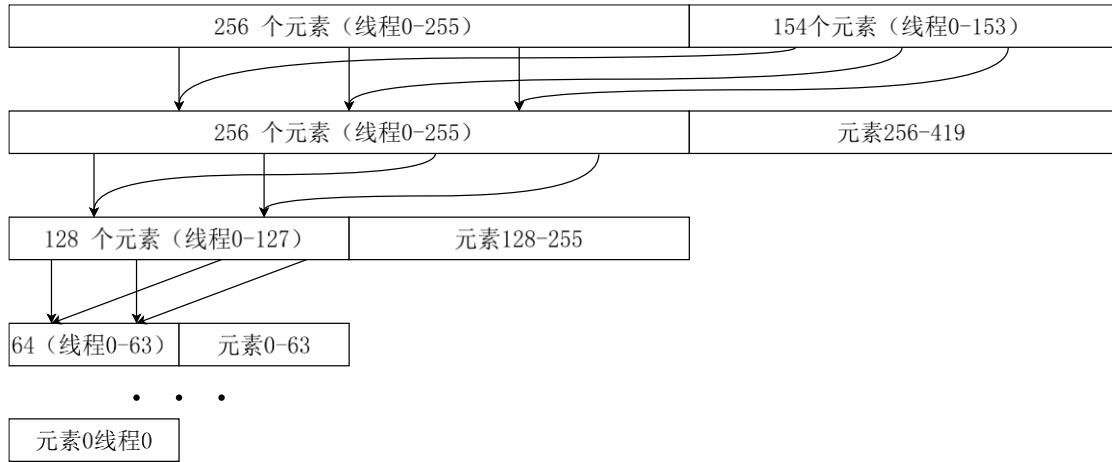


图 4.6 归约方法示意图

频偏估计计算完成后，就需要利用计算得到的结果对参考通道和监测通道的数据进行相位旋转以实现频偏补偿。对于同步后的采样数据 $x[n]$ ，按照表达式(4.1)的方法进行频偏补偿：

$$y[n] = x[n] \cdot e^{-j(2\pi\Delta f n T)} \quad (4.1)$$

其中 $y[n]$ 为补偿后的结果， Δf 为计算得到的频偏， T 为 DTMB 信号的符号周期。这些对每个点进行频偏补偿的操作都相互独立，所以十分适合用 GPU 进行加速计算实现。GPU 使用 CUDA 的硬件内置指令__sincosf，可以在少量的时钟周期内同时计算出给定相位的正弦和余弦值，极大提升运算速度。

分别在 CUDA 项目和 Matlab 项目上对同一数据进行频偏估计与补偿，以验证 GPU 加速实现频偏估计与补偿的性能。采用的数据为 2020 年洛阳采集到的参考方向指向信号发射站的实验数据，所得第一次批处理的结果如表 4.2 所示

表 4.2 GPU 和 Matlab 进行频偏估计与补偿的结果

运行平台	方法 0 平均值	方法 0 标准差	方法 1 平均值	方法 1 标准差
CUDA	-36.45Hz	0.212Hz	-36.46Hz	0.188Hz
Matlab	-36.45Hz	0.212Hz	-36.46Hz	0.188Hz
运行平台	频偏估计耗时	频偏补偿耗时	总耗时	
CUDA	190.8μs	16.2μs	207.0μs	
Matlab	13.4ms	19.9ms	33.3ms	

两个项目都是对参考通道 178 个数据帧进行频偏估计，对参考通道和监测通道总共 534 个数据帧的数据进行频偏补偿。其中方法 0 是利用两信号帧的全 PN420 序列来进行频偏估计，而方法 1 利用两个信号帧的后 156 点 PN420 序列进行频偏估计。方法 1 虽然参与计算的互相关长度缩短，但后段 PN 序列受多径影响较小。由表 4.2 可以看出，利用 GPU 进行频偏估计运算的结果中，方法 0 和方法 1 的频偏均值和标准差结果都是几乎相同的，实际上频偏均值的相对误差都在百万分之三以内。Matlab 项目中的数据计算形式都是 Complex Double 型，而 CUDA 项目中的数据形式是 cuFloatComplex，为单精度浮点型复数，这会导致后者精度要略逊于前者，但因为数据形式导致的误差对整个 DTMB 参考信号重构处理结果影响可以忽略不计。除此之外，可以看出频偏估计和频偏补偿中，GPU 运算速度都要显著快于 CPU，这是因为频偏估计和补偿是一直在 GPU 上运行的，没有进行非常耗时的内存拷贝操作。同时表中结果也验证了 GPU 加速运算的准确性和有效性。

4.5 信道估计方法的 GPU 加速实现

为了满足 DTMB 参考信号重构系统对实时性的高要求，本研究利用 CUDA 并行计算方式分别对时域相关信道估计、频域信道估计和改进的时域相关信道估计算法进行了硬件加速实现。由于最小二乘信道估计方法运行耗时最长且效果与改进的时域相关方法几乎一致^[15]，所以这里不提及该方法的实现。

4.5.1 原时域相关信道估计方法的 GPU 实现

在第三章信道估计方法中提到，该算法主要包括两个核心步骤：接收信号与本地 PN 序列的循环相关运算与旁瓣干扰迭代消除。为了减少每次批处理要进行的多次信道估计任务中的显存分配和释放的时间开销，CUDA 项目中通过 H_Estimate_GPU_Cache 缓存结构体来一次性分配所有必需的设备内存，包括输入信号序列、PN 序列、相关结果、幅值数组、峰值索引数组及临时信道响应缓冲区所占用的显存，直到该次批处理中循环性重构完成时销毁。在带缓存的接口 h_estimate_gpu_with_cache 中，核函数直接使用预分配的缓存指针，省略了设备内存的动态申请步骤。

首先 GPU 计算接收的 255 点信号序列 $r[n]$ 与本地 PN 序列 $PN[n]$ 的循环相关，该运算在 GPU 上通过核函数 compute_ht_kernel 实现并行化计算。每个线程

块负责计算一个输出延迟 τ 的相关值 $ht[\tau]$ 。信道估计长度 M 为 255，启动 255 个线程块，每个线程块配置 64 个线程。为避免全局内存的原子操作冲突，线程块内使用共享内存进行两阶段规约：首先各线程将局部累加和写入共享内存数组，随后通过规约（循环折半求和）将块内 64 个部分和归约为最终的相关值，由线程 0 写入全局内存，这与图 4.6 中所表示的归约方法一致。

得到的相关结果 $ht[\tau]$ 为复数数组，计算其幅值以判别其有效多径。幅值计算的核函数 `h_estimate_compute_abs_kernel` 中每个线程单独计算一个元素的幅度 $A = \sqrt{Re^2 + Im^2}$ ，然后将幅度结果传回主机端。主机端找到所得相关结果最大值 A_{max} 后，设定 $0.05A_{max}$ 为判别门限，大于 $0.05A_{max}$ 的值为有效多径，将这些有效多径按从大到小的顺序排列之后，先将所有相关结果除以信道长度 $M=255$ ，得到初始信道估计结果 $h^{(0)}$ ，然后依照峰值能量从强到弱的顺序为 $h^{(0)}[\tau_0]$ ， $h^{(0)}[\tau_1] \dots h^{(0)}[\tau_{N-1}]$ ，将第 i 个峰值径的能量均摊至其余所有非峰值位置上，迭代更新信道响应，如式（4.2）所示：

$$h^{(i)}[k] = h^{(i-1)}[k] + \frac{h^{(i-1)}[\tau_i]}{M}, \forall k \neq \tau_i \quad (4.2)$$

利用 CUDA 项目和 Matlab 项目计算得到的时域相关信道估计结果分别如图（4.7）中左图和右图所示，所采用的数据均为 2020 年在洛阳采集到的参考方向指向发射站的数据。

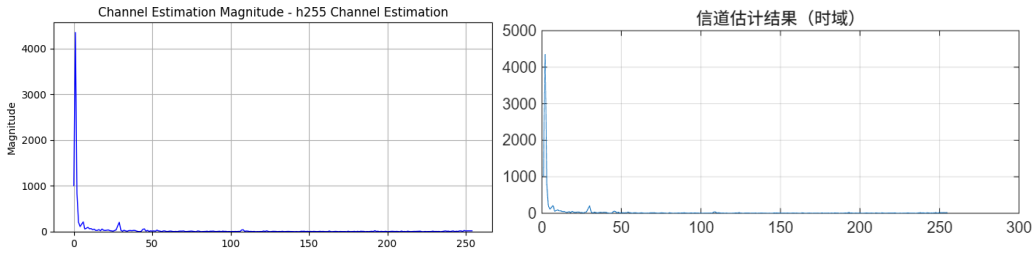


图 4.7 CUDA 项目与 Matlab 项目时域相关信道估计结果

可以看出 Matlab 和 CUDA 项目的时域相关信道估计结果几乎是一致的，Matlab 和 CUDA 项目中主径峰值为 4308，除主径与旁瓣外的最强多径峰值都为 221。这验证了利用 GPU 实现信道估计方式的可靠性。虽然时域相关信道估计方法的旁瓣干扰消除步骤中，信道响应循环迭代在第一次信道响应结果更新后就不需要额外再分配内存，但是这种需要依赖前一次结果来进行更新的计算方式难以实现并行计算，也就无从发挥 GPU 的大规模并行计算能力。

4.5.2 频域信道估计方法的 GPU 实现

由于时域相关信道估计方法中的数据帧头和本地帧头的循环互相关计算的时间复杂度为 $O(N^2)$ ，为了加快信道估计的速度，频域信道估计方法直接在频域上将数据帧头和本地帧头相除，得到频域的信道响应。这种方法使用 FFT 和 IFFT 的计算来代替循环互相关，可以将时间复杂度降低到 $O(N\log(N))$ 。在计算过程中利用 CUDA 的二维线程网格按元素并行执行复数除法时，为保证数值稳定性，核函数内部加入了分母极小值校验（ $\text{denom} > 1e-10f$ ），当发生除零风险时安全输出零值。在 GPU 批处理中的频域信道估计算法如图（4.8）所示。

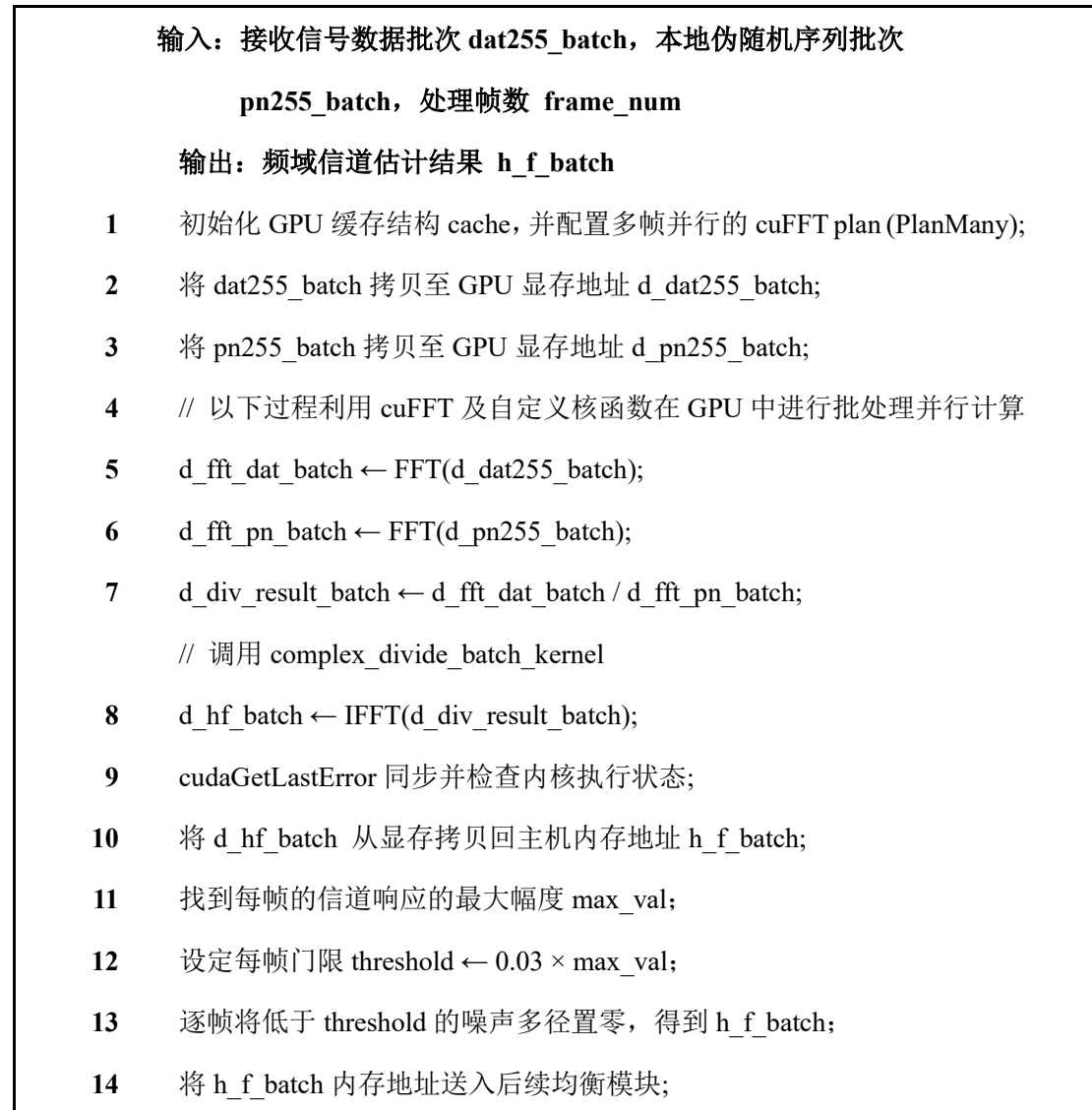


图 4.8 频域信道估计算法的 GPU 实现

利用 CUDA 项目和 Matlab 项目计算得到的频域信道估计结果分别如图(4.9)中左图和右图所示，其中纵坐标为信道响应幅度。

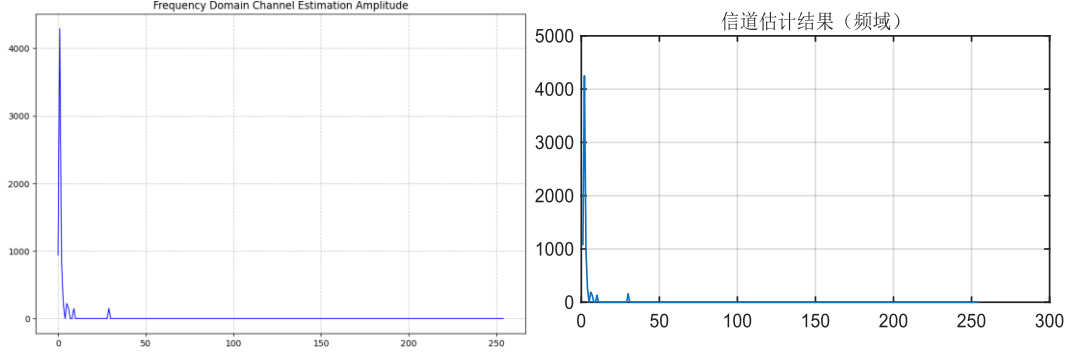


图 4.9 CUDA 项目与 Matlab 项目频域信道估计结果

可以看出二者频域信道估计结果几乎是一致的，Matlab 项目和 CUDA 项目中频域信道估计结果主径峰值为 4251，除主径与旁瓣外的最强多径峰值为 159，CUDA 项目中为 160，同样验证了利用 GPU 实现信道估计方式的可靠性。相比于时域相关信道估计方法，频域方法中主径响应结果很相近，但多径响应的结果有较大的差异，其他位置的响应则为 0。虽然频域信道估计方法实现简单且计算速度快，但其使用硬截断的方式，在噪声较大时难以分辨其与较弱多径的区别，因此只适合在噪声远小于多径干扰的条件下使用。

4.5.3 改进的时域相关信道估计方法的 GPU 实现

原时域相关信道估计方法中的本地 PN 序列和接收信号 PN 序列的循环互相关计算时间复杂度较高，为 $O(N^2)$ 。可以将其转化为这两个序列的频域共轭相乘以降低时间复杂度至 $O(N \log(N))$ 。设复数序列 r 的频域结果为 R ，本地 PN 序列 p 的频域结果为 P 。对 $r[n]$ 进行反褶与取模后再取共轭，得到表达式 (4.3)：

$$r^{\dagger}[n] = r^*[-n \bmod N] \quad (4.3)$$

设 $n = (-k) \bmod N$ 。对于循环互相关计算，有：

$$\begin{aligned} r_{xcorr}[m] &= \sum_{n=0}^{N-1} p[(n+m) \bmod N] \cdot r^*[n] \\ &= \sum_{n=0}^{N-1} p[(m - (-n)) \bmod N] \cdot r^*[n] \\ &= \sum_{k=0}^{N-1} p[(m - k) \bmod N] \cdot r^*[-k \bmod N] \\ &= p[m] \odot r^{\dagger}[m] \end{aligned} \quad (4.4)$$

其中符号 $\circledast$ 表示循环卷积。由于序列循环卷积的 DFT 等于各自 DFT 的逐点乘积，所以有：

$$\text{FFT}\{r_{\text{corr}}[n]\} = \text{FFT}\{p[n] \circledast r^\dagger[n]\} = P[k] \cdot R^\dagger[k] \quad (4.5)$$

根据反褶与共轭的特性，有：

$$R^\dagger[k] = \text{FFT}\{r^*[-n \bmod N]\} = R^*[k] \quad (4.6)$$

将式（4.6）和（4.5）代入表达式（4.4），最终得到：

$$r_{\text{corr}}[n] = \text{IFFT}\{P[k] \cdot R^*[k]\} \quad (4.7)$$

这样转换可以使得其运算速度和频域信道估计算法几乎一样快。除此之外，原方法中的逐次旁瓣消除过程可转化为对相关耦合关系的直接求解。本文采用直接求解线性方程的方式，将因 PN 序列的自相关特性导致的旁瓣干扰一次性计算出来得到修正项，使这种干扰消除只需要一次简单的点除和 M 次相加。本文给出在 GPU 上实现的改进时域相关信道估计算法，如图（4.10）所示。

输入：本地 PN 序列 **pn255**，接收信号的帧头 **dat255**，帧数 **frame_num**

输出：信道估计结果 **hr**

- 1 初始化 HEstimateImprovedGPUCache 缓存结构体，分配设备内存 **d_dat255_batch**, **d_pn255_batch**, **d_fft_dat_batch**, **d_fft_pn_batch**, **d_conj_mul_result_batch**, **d_hr_batch**;
- 2 配置 cuFFT 批量 FFT 和 IFFT 计划 **fft_plan_batch**, **ifft_plan_batch**;
- 3 将 **dat255** 与 **pn255** 拷贝至 GPU 全局内存;
- 4 对 **frame_num** 个接收数据 **dat255** 执行批量前向 FFT 得到 **d_fft_dat_batch**;
- 5 对 **frame_num** 个 **pn255** 数据执行批量前向 FFT 得到 **d_fft_pn_batch**;
- 6 **d_fft_dat_batch** 与 **d_fft_pn_batch** 共轭相乘得到 **d_conj_mul_result_batch**;
- 7 **d_hr_batch** ← IFFT(**d_conj_mul_result_batch**); //批量 IFFT
- 8 **cudaDeviceSynchronize**; //设备同步
- 9 将原始信道估计结果 **d_hr_batch** 拷贝回主机内存并归一化缩放得到 **hr_batch**;
- 10 找到 **hr_batch** 最大值 **max_val**，计算有效多径门限 **threshold = 0.05*max_val**;
- 11 筛选有效径数 **n** 并计算其累加值 **sum_RPr**，计算修正补偿 **correction = sum_RPr / (M + 1 - n)**;
- 12 得到补偿后的信道估计：**hr = hr_raw + correction**。

图 4.10 改进的时域相关信道估计算法 GPU 实现

利用 CUDA 项目和 Matlab 项目计算得到的改进的时域相关信道估计结果分别如图（4.11）的左图和右图所示。

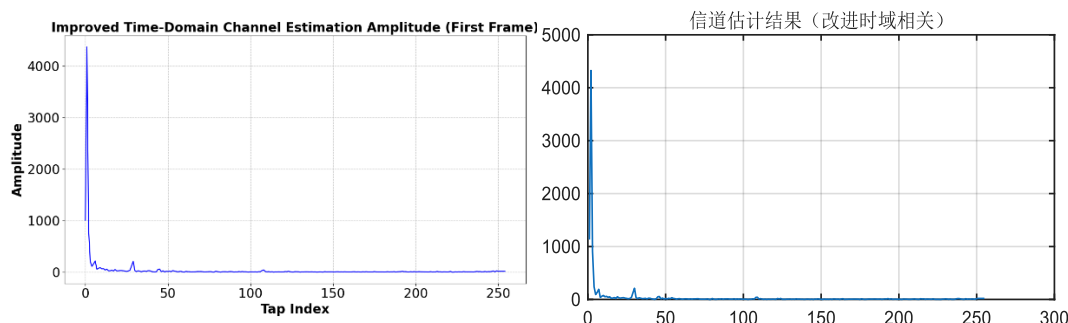


图 4.11 CUDA 项目与 Matlab 项目改进的时域相关信道估计结果

二者信道估计结果中 Matlab 项目与 CUDA 项目中的主径幅度都为 4325，除主径旁瓣之外的最强多径都为 211，这表明 CUDA 项目成功实现了改进的时域相关信道估计。对比三种信道估计方法的结果，可见改进的时域相关信道估计方法所得到的结果比频域方法的更接近原时域方法结果。事实上，这三种信道估计结果中，Matlab 项目和 CUDA 项目的运算结果绝对值的误差都在 0.2 以内。在后文的实测性能结果分析这一节，将详细展示三种信道估计方法的计算效率和效果。

4.5.4 实测性能结果分析

利用在 2020 年洛阳参考信号多径采集到的数据进行信道估计，在 CUDA 项目和 Matlab 项目运行一次批处理，每次批处理 180 数据帧，三种信道估计方法分别消耗的总时间如表 4.3 所示。

表 4.3 三种信道估计方法在 Matlab 和 GPU 上运行消耗时间

运行项目	消耗时间
GPU 时域相关	599.5 μ s
GPU 频域估计	225.0 μ s
GPU 改进时域	190.8 μ s
Matlab 时域相关	74.8ms
Matlab 频域估计	12.0ms
Matlab 改进时域	13.6ms

可以看出无论是在 Matlab 还是在 CUDA 项目中，频域信道估计方法和改进的时域相关信道估计方法耗时都要显著短于原时域相关信道估计方法。在 CUDA 项目中，改进时域相关方法耗时要短于频域估计方法。相较于难以利用 GPU 实

现并行运行的旁瓣干扰消除方法，进行门限截断或直接求解线性方程的方法都可以节约大量的运行时间。CUDA 项目中的三种信道估计算法的运行速度相比于 Matlab 项目都有明显的提升。本文项目中信道估计结果不需要传回主机端，而是直接用于后续的参考信号提纯，因而利用 GPU 进行信道估计可以极大提升整个参考信号重构系统的运行速度。

重构参考信号的质量决定了 DBPR 系统检测通道的杂波抑制效果，而信道估计结果会直接影响到得到的重构参考信号的质量。对于参考信号的质量评估方式，这里选用调制误差率（Modulation Error Rate, MER）来衡量参考信号重构后的调制质量^[27]。MER 是理想星座点平均功率与误差矢量平均功率之比的对数表示（单位：dB），可以看作是“信噪比”在数字调制中的体现。其计算方式如式（4.8）所示：

$$MER = \frac{\sum_{n=1}^N |I_{ideal,n} + j Q_{ideal,n}|^2}{\sum_{n=1}^N |(I_{measured,n} - I_{ideal,n}) + j (Q_{measured,n} - Q_{ideal,n})|^2} \quad (4.8)$$

其中 I_{ideal} 和 Q_{ideal} 为理想星座映射对应的同相和正交方向的值， $I_{measured}$ 和 $Q_{measured}$ 为实际接收符号的同相和正交方向的值。利用经过 Brazil A 信道传输、64QAM 映射方式的 DTMB 信号来比较不同信噪比（SNR）下三种信道估计方法的得到的星座图的 MER，结果如图（4.12）所示：

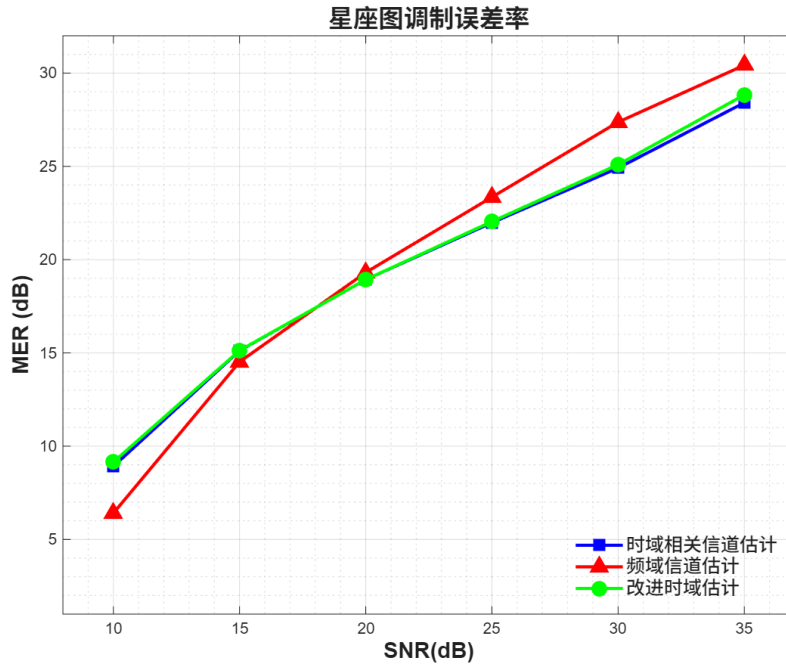


图 4.12 三种信道估计方法的调制误差率

如图所示，在信号的信噪比较低时，频域信道估计方法得到的星座图 MER 最低；在信噪比较高时，频域方法的 MER 最高。在低 SNR 时，频域信道估计及其容易将较大噪声判定位多径；高 SNR 时，由于 Brazil-A 模拟信道的最小多径幅度约为主径的 0.09，远大于硬判决阈值 0.03，使得几乎所有杂波都能得到有效的抑制，因此频域估计方法得到的结果与时域相关估计结果有上述差别。实际环境信道往往存在大量及其微弱的多径，因此该结果仅供参考。在后文系统所用的数据为在洛阳采集的实测数据，这个实测数据里就存在大量微弱多径。改进的时域相关信道估计方法得到的星座图则在 10dB ~ 35dB 的 SNR 条件下均略强于原时域相关信道估计方法。当信噪比无限扩大时，三种方法的 MER 也只能接近 33.5dB。这是因为在 Brazil A 信道影响下，残余多径干扰和信道估计误差成为了影响星座图 MER 的主要因素^[28,29]。

4.6 参考信号提纯的 GPU 加速实现

参考信号提纯是整个 DTMB 参考信号重构流程中除降采样之外计算量最大的模块，在 GPU 上实现这个模块可以极大的提升整个参考信号重构系统的运行速度。提纯步骤包括利用信道估计结果进行循环性重构、频域均衡与组帧。

对于循环性重构步骤，类似于改进的时域信道估计方法的计算互相关的思路，将其转化到频域上进行运算要显著快于直接在时域上卷积运算。所以在 CUDA 项目中，conv_kernel 卷积核实际上进行的操作是经过 FFT 后共轭相乘再转回时域，然后利用得到的卷积结果对帧体进行符号间干扰消除，最后提纯出经干扰消除与均衡后的帧体，再将本地 PN 序列拼接和数据帧，得到提纯后的参考信号。GPU 实现方法如图（4.13）所示。

输入：本地 PN 序列 pn420_bz，接收信号的帧数据 dat，帧数 frame_num；

输出：提纯后的信号 refcleardata。

- 1 初始化 GPU 缓存结构体 RefClearGPUCache，分配设备内存 d_dat，
d_frame, d_g1_batch, d_g2_batch, d_data_jun_batch, d_refclrdata；
- 2 将 dat, pn420_bz 拷贝至 GPU 全局内存 d_dat, d_pn420_bz；
- 3 根据 GPU 内存数据批量利用信道估计方法得到 d_hr_mul_result_batch；
- 4 从 d_pn420_bz 本地序列中得到循环前缀 d_pn1 与循环后缀 d_pn2；

- 5 对 frame_num 个 d_pn1 和 d_pn2 分别与信道估计结果 d_hr_mul_result_batch 卷积得到结果 d_g1_batch 与 d_g2_batch;
- 6 使用 subtract_and_add_batch_kernel 核函数对从 d_dat 提取到的帧体进行符号间干扰消除, 得到 d_data_jun_batch;
- 7 对 d_data_jun_batch 执行批量 FFT, 得到 d_data_in_fft_freq_batch, 再除以信道估计结果 d_hr_mul_result_batch 得到均衡后数据 d_freq_refclldata;
- 8 对均衡后结果进行批量 IFFT 核归一化并组帧, 得到重构后信号 d_refclldata;
- 9 设备同步 cudaDeviceSynchronize;
- 10 将设备端数据传回主机端得到重构后信号 refclldata。

图 4.13 参考信号提纯的 GPU 实现方法

利用 GPU 实现的参考信号提纯得到的重构后数据的星座图如图 (4.14) 所示。其中信道估计方法为改进的时域相关信道估计, 使用的数据为洛阳多径实验数据。

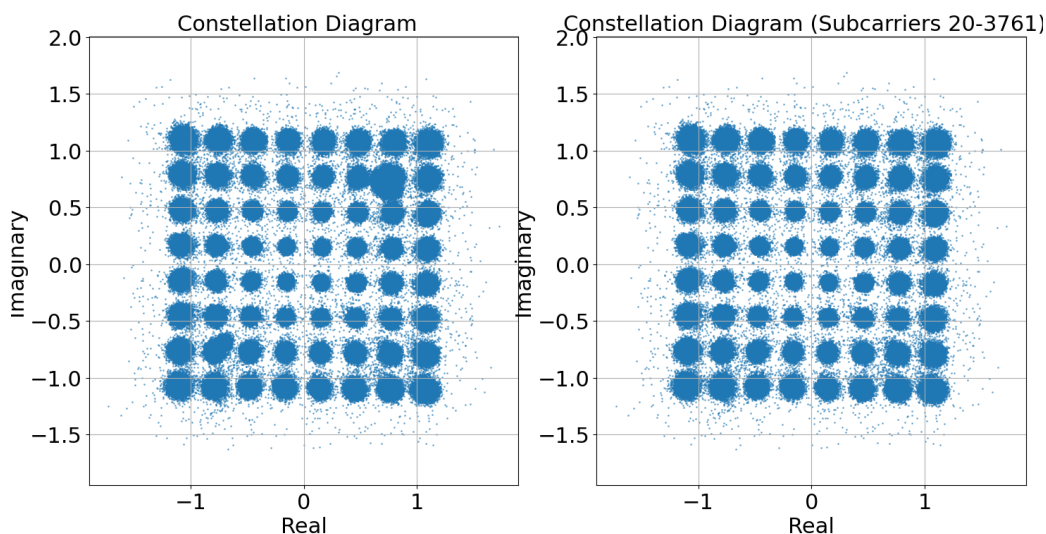


图 4.14 GPU 实现的重构参考信号的星座图

左图是第一次批处理中所有得到数据帧的全部子载波数据, 右图是所有数据帧中去掉低频和高频总共 38 个子载波后的数据。由于 DTMB 信号中前 18 和最后 18 个子载波是离散导频, 所以在归一化前坐标为 (5, 5) 与 (-5, -5) 位置会出现这些离散导频的符号。

为了比较参考信号重构的性能, 分别对 CUDA 项目和 Matlab 项目中利用三种信道估计方法提纯得到调制误差率的结果和这两个项目运行消耗时间(包含信道估计与三路通道信号导出时间)如表 4.4 所示。

表 4.4 Matlab 和 GPU 上参考信号提纯性能

所用平台与方法	调制误差率（dB）	消耗时间（ms）
GPU 时域相关	25.21	12.98
GPU 频域估计	22.51	1.24
GPU 改进时域	25.13	1.08
Matlab 时域相关	25.24	1629.89
Matlab 频域估计	22.53	119.19
Matlab 改进时域	25.16	144.51

可以看出 CUDA 项目上和 Matlab 项目的调制误差比结果十分接近，由于 GPU 上数据计算类型是单精度浮点型复数，而 Matlab 上是双精度浮点型复数，所以 Matlab 项目上得到的调制误差率效果要略好一些。但是 GPU 进行参考信号提纯的速度要显著快于 Matlab 项目，这表明本项目实现了 GPU 进行参考信号重构加速的效果。

4.7 DTMB 参考信号重构的硬件实时化实现仿真

在系统无发送端，离线处理本地信号时，对于每次 100ms 长度、180 数据帧的三路 DTMB 信号的处理，参考信号重构项目平均运行耗时约为 40ms，几乎所有批处理消耗的时间都集中在 35ms~55ms 内，运行耗时结果如表 4.5 所示。这理论上能够满足在 100ms 批处理周期下参考信号重构的实时化处理。

表 4.5 DTMB 参考信号重构各模块耗时统计

运行项目	消耗时间
降采样（包含数据传输）	26.11ms
信号同步	331.1μs
频偏估计	190.8μs
频偏补偿	16.2μs
信号提纯（包含信道估计）	1.06ms
重构数据导出	12.28ms

基于此结果设计模拟的 DTMB 参考信号接收与重构系统。本系统拥有模拟的 DTMB 信号的接收端和信号处理端，信号接收平台以每 100ms 传输 180 数据帧的速度将一路参考通道和两路监测通道的 DTMB 信号存储到内存中，处理端则实时读取这三路信号并进行批处理。信号处理初始化界面如图（4.16）所示：

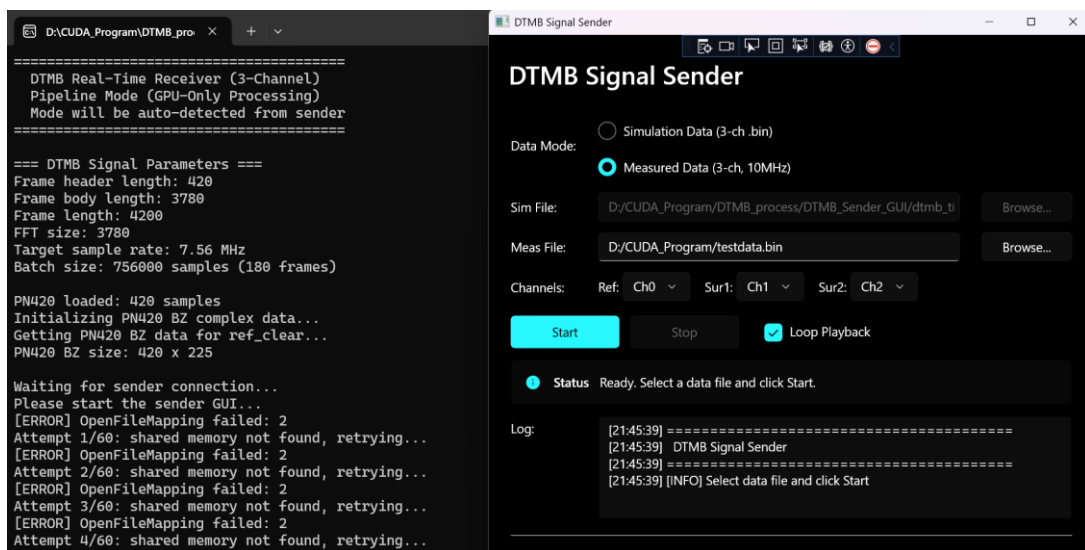


图 4.15 信号处理初始化界面

接收端使用 WinUI3 框架进行 UI 设计。首先读取准备传输的数据文件，由于 DBPR 系统实际应用中接收端会使用光模块将信号传送到信号处理端，这里选择在主机端创建共享内存区域，用于信号的实时化传输。接收端支持单次传输所有数据和循环传送数据的功能，且信号传输可以随时中断与重启，且精确的控制每 100ms 发送每路 180 数据帧的两路采用 64QAM 星座映射方式、PN420 帧头模式的 DTMB 信号。处理端在加载好本地 PN 序列和找到共享内存区域后即可开始实时化的接收与处理数据，对于没能及时处理的数据会先存放到缓冲区里，由于处理端平均处理时间为 58ms，远小于 100ms 所要求的批处理周期，所以只要接收端缓冲区能存放三路 180 帧的数据，则几乎不会出现缓冲区数据溢出的问题。处理过程中输出的信号同步结果如图（4.17）所示。

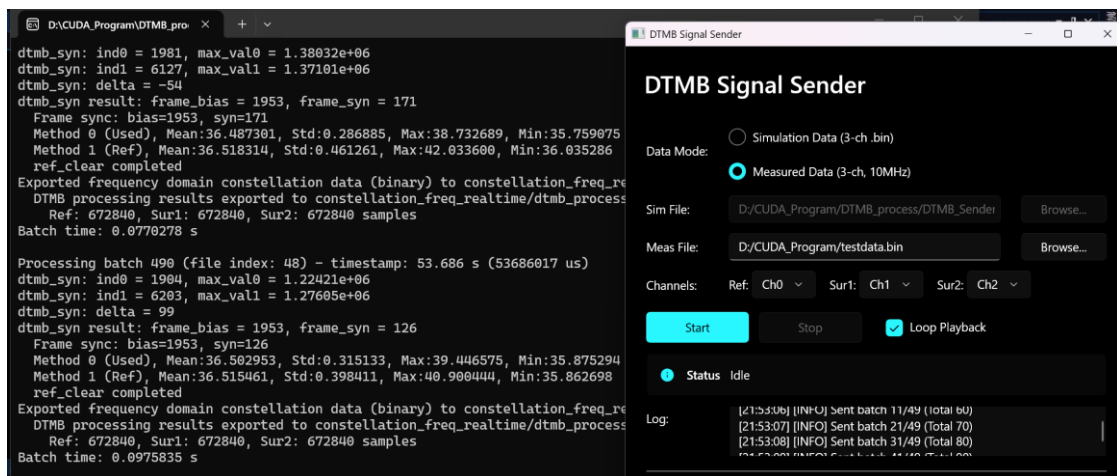


图 4.16 实时信号处理系统运行界面

以图中所示第 489 次批处理为例,可以看出信号同步模块中信号偏移为 1953, 帧号为 171, 频偏估计模块(方法 0)中的频偏结果为 36.48, 频偏标准差为 0.287, 而第 490 次批处理中帧号变为了 126, 由于每次批处理 180 帧, 而帧号周期为 225, 所以这些数据证明了该系统运行结果是正确的。此外, 系统会实时的生成重构后的参考信号与监测信号的二进制数据文件, 从而可以很方便的进行后续的杂波抑制、距离多普勒谱生成和目标探测。

将该系统输出的参考通道和监测通道数据进行杂波抑制与距离多普勒谱生成, 得到结果如图 (4.18a) 所示, Matlab 项目得到的结果如图 (4.18b) 所示。可以看出二者的结果几乎是一致的, 峰值位置都在 119 距离元、-2 多普勒元。CUDA 项目中峰值幅度为 -20.54dB, 基底幅值为 -66.50dB, 最强目标相对于基底的幅度为 45.96dB; Matlab 项目中峰值同样为 -20.54dB, 基底幅值为 -66.49dB, 相对幅度为 45.95dB, 二者的结果十分接近。

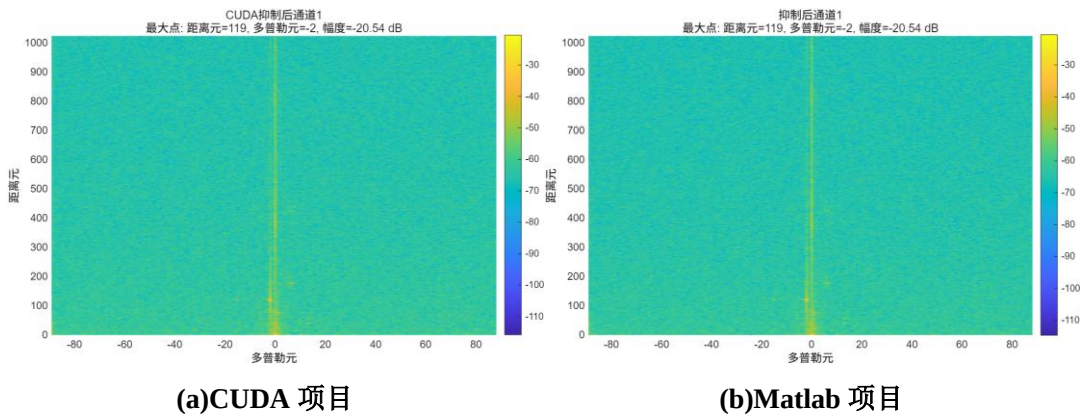


图 4.17 距离多普勒谱

这说明利用 CUDA 项目输出的重构参考信号结果得到的距离多普勒谱是有效的。最后进行性能分析。对于本参考信号重构系统实时批处理过程, 通过性能分析工具 NVIDIA Nsight System 运行参考信号重构系统, 得到的 CUDA GPU Summary 统计结果中部分性能分析结果如图 (4.18) 所示。

Time	Total Time	Instances	Avg	Med	Min	Max	StdDev	Category	Operation
42.1%	479.695 ms	996	481.621 μ s	15.568 μ s	960 ns	5.578 ms	919.936 μ s	MEMORY_OPER	[CUDA memcpy Device-to-Host]
34.8%	396.381 ms	332	1.194 ms	807.463 μ s	0 ns	7.449 ms	1.529 ms	MEMORY_OPER	[CUDA memcpy Host-to-Device]
7.2%	81.928 ms	249	329.029 μ s	188.414 μ s	97.279 μ s	2.316 ms	515.012 μ s	CUDA_KERNEL	void regular_fft_factor<(unsigned int)125, EPT<
2.8%	31.503 ms	83	379.554 μ s	188.414 μ s	183.294 μ s	2.303 ms	584.888 μ s	CUDA_KERNEL	void regular_fft_factor<(unsigned int)64, EPT<
1.7%	19.219 ms	83	231.557 μ s	98.303 μ s	96.255 μ s	1.561 ms	399.807 μ s	CUDA_KERNEL	packFrequencyDomain_batch(const float2 *, fl
1.7%	18.966 ms	249	76.170 μ s	38.912 μ s	28.672 μ s	538.617 μ s	106.095 μ s	CUDA_KERNEL	void vector_fft<(unsigned int)3780, EPT<(unsig
1.5%	16.822 ms	83	202.675 μ s	98.303 μ s	96.254 μ s	1.429 ms	340.340 μ s	CUDA_KERNEL	freq_compensate_multi_channel_inplace_kern
1.4%	15.394 ms	83	185.465 μ s	91.135 μ s	86.015 μ s	1.101 ms	267.500 μ s	CUDA_KERNEL	void regular_fft_factor<(unsigned int)32, EPT<
1.3%	15.361 ms	83	185.070 μ s	92.159 μ s	87.039 μ s	1.080 ms	267.850 μ s	CUDA_KERNEL	void regular_fft_factor<(unsigned int)27, EPT<
1.3%	14.566 ms	83	175.496 μ s	103.423 μ s	99.327 μ s	926.708 μ s	224.582 μ s	CUDA_KERNEL	void regular_fft_factor<(unsigned int)7, EPT<
0.8%	8.831 ms	166	53.198 μ s	36.864 μ s	24.575 μ s	497.659 μ s	86.528 μ s	MEMORY_OPER	[CUDA memcpy Device-to-Device]
0.6%	6.624 ms	415	15.961 μ s	10.240 μ s	7.167 μ s	110.591 μ s	18.316 μ s	CUDA_KERNEL	void vector_fft<(unsigned int)1024, EPT<(unsig
0.5%	5.264 ms	83	63.425 μ s	36.863 μ s	35.839 μ s	228.350 μ s	47.265 μ s	CUDA_KERNEL	normalizeFFT(float2 *, int, float)

图 4.18 部分性能分析结果 (CUDA GPU Summary)

可以通过图（4.18）看出对于 83 次批处理中，主机端与设备端之间的数据传输占据了整个 GPU 活动过程的约 78%的时间，真正使用核函数进行计算所消耗的时间只占用 GPU 活动过程时间的约 22%，每次批处理中设备端都需要向主机端传送 12 次数据，主机端需要向设备端传送 4 次数据，说明 GPU 加速计算的效果极其显著，项目运行速度瓶颈在于主机端于设备端之间的数据传输速度。下面通过 Timeline View 详细分析其中一次批处理中硬件运行情况。



图 4.19 Timeline View 分析结果

从图（4.19）可以看出，每次批处理中 SM 计算单元活跃时间约为 2ms，这说明主要的计算过程耗时极短。在第 3s 中+216ms 与+224ms 的大致时间尺度内，该系统在执行降采样中除 CPU 预处理外的过程，CPU 读取数据预处理阶段则在约 3s 中+205ms 至+216ms 的时间尺度；从 3s+225ms 开始、持续约 2ms 的过程中，该系统完成了信道估计、频偏估计与补偿、信道估计与参考信号提纯功能；在之后的约 8ms 内，系统分别在参考信号的频域帧体信号、参考信号的时域完整信号、两路监测通道时域信号的导出。相较于平均每次批处理需要消耗约 990ms 的 CPU 实现方法，本文中 CUDA 项目相对于 Matlab 项目拥有约 25 倍的加速比，实现了面向实时处理的硬件加速原型验证。

5 总结与展望

5.1 全文总结

本文探讨了 DTMB 外辐射源雷达参考信号重构方法，并基于这种方法设计了利用 GPU 硬件实现了 DTMB 参考信号实时化重构系统，且利用杂波抑制后的距离多普勒谱结果验证了 GPU 实现参考信号重构的可靠性。

传统的利用 CPU 进行参考信号重构的方式运行速度较慢，无法满足实际应用中 DBPR 系统实时化运行的功能要求。基于此设计利用 GPU 硬件进行参考信号重构，在 GPU 上实现信号同步、频偏估计与补偿、信道估计与参考信号提纯的功能，将主要的运算都移动到 GPU 上实现，整个系统相对于 CPU 实现方式有大约十倍的加速效果，在本文平台下可满足实时批处理要求。

由于原时域相关信道估计方法的运行速度很慢，循环卷积计算时间复杂度高，迭代消除难以并行化运行，这两个步骤都会消耗大量运行时间。为了解决这个问题，在 CUDA 项目中实现了改进的时域相关信道估计方法，用频域相乘代替时域循环卷积，并且利用直接求解线性方程的方式避免了迭代方式的杂波消除，极大缩短了运行时间，且信道估计效果与原时域相关信道估计方法的效果几乎一致。

5.2 后续工作展望

本项目已经完成了对 DTMB 参考信号的实时化重构，但还有功能与诸多性能优化方面的问题需要后续的完善。由于本项目使用的设备是笔记本电脑，会存在主机内存和显存之间的数据传输速度受限于 PCIe 总线带宽的情况，使得设备端在进行读取采集到的数据时消耗大量的时间。所以对于更多通道数据的降采样处理，当前平台难以实时化运行。此外，由于该项目只生成了一路重构的参考信号和两路监测通道的信号，会影响后续的空域杂波抑制与波束形成的效果。根据图（4.18）可以看出，在降采样处理过程中，大量的数据还是会在主机端和设备端之间传输，这会消耗大量的时间。除此之外，生成的重构后参考信号以及两路监测通道信号的导出速度会受限于 PCIe 通道速度以及主机端的处理速度，后续可以设计为直接在本项目中进行空域杂波抑制与波束形成运算；参考信号提纯模块中没能完全占用所有 GPU 资源进行计算，后续需要进一步优化这些模块以实现更高的加速比效果。

参考文献

- [1] Griffiths H D, Baker C J. Passive coherent location radar systems. Part 1: Performance prediction[J]. IEE Proceedings - Radar, Sonar and Navigation, 2005, 152(3): 153-159.
- [2] 杨知行. 时域同步正交分频复用接收机系统[D]. 北京: 清华大学, 2007.
- [3] Xie D, Yi J, Wan X, et al. Masking effect mitigation for FM-based passive radar via nonlinear sparse recovery[J]. IEEE Transactions on Aerospace and Electronic Systems, 2023, 59(6): 8246-8262.
- [4] Mata-Moya D, Del-Rey-Maestre N, Hernandez-Fernandez C, et al. Improvement of DVB-T Based Passive Radar Detection Using Multi-Frequency Diversity[C]//2025 IEEE International Conference on Signals and Systems (ICSigSys). IEEE, 2025: 60-65.
- [5] Taylor A, Poullin D. Experimental UAV detection using 4G-LTE-based passive Radar[C]//2023 IEEE International Radar Conference (RADAR). IEEE, 2023: 1-6.
- [6] Zheng B, You C, Mei W, et al. A survey on channel estimation and practical passive beamforming design for intelligent reflecting surface aided wireless communications[J]. IEEE Communications Surveys & Tutorials, 2022, 24(2): 1035-1071.
- [7] Attalah M A, Laroussi T, Gini F, et al. Range-Doppler Fast Block LMS algorithm for a DVB-T-based Passive Bistatic Radar[J]. Signal, Image and Video Processing, 2019, 13: 27-34.
- [8] 李中余, 皮浩卓, 李俊奥, 等. 双基 SAR 空时自适应 ANM-ADMM-Net 杂波抑制技术[J]. 雷达学报, 2024, 14(5): 1196-1214.
- [9] 万显荣, 程熠瑶, 易建新, 等. DTMB 外辐射源雷达参考信号重构信道估计新方法[J]. 电子与信息学报, 2017, 39(5): 1044-1050.
- [10] 赵志欣, 戴文婷, 陈鑫, 等. 基于深度神经网络的正交频分复用波形外辐射源雷达参考信号重构[J]. 电子与信息学报, 2021, 43(9): 2735-2742.

- [11] 史嫻嫻, 魏彬, 李爱华, 等. 5G-Advanced 核心网运行智能架构演进, 关键技术及应用研究[J]. 电信科学, 2024, 40(12): 146-162.
- [12] 刘乔寿, 周雄, 刘爽, 等. 基于深度强化学习的 OFDM 自适应导频设计[J]. 通信学报, 2023, 44(9).
- [13] 李晓波, 关欣, 仲利华, 等. 基于 GPU 的外辐射源雷达信号处理实时实现方法[J]. 系统工程与电子技术, 2014, 36(11): 2192-2198.
- [14] 邵启红. 低频段外辐射源雷达信号处理若干关键技术研究[D]. 武汉大学, 2016.
- [15] 张勋. 数字广播外源雷达参考信号重构关键技术研究[D]. 武汉: 武汉大学, 2021.
- [16] Zheng Z W, Yang Z X, Pan C Y. Novel synchronization for TDS-OFDM-based digital television terrestrial broadcast systems[J]. IEEE Transactions on Broadcasting, 2004, 50(2): 148-153.
- [17] Bai D, Nam W, Lee J, et al. Comments on “A Technique for Orthogonal Frequency Division Multiplexing Frequency Offset Correction”[J]. IEEE Transactions on Communications, 2013, 61(5): 2109-2111.
- [18] Van de Beek J J, Sandell M, Borjesson P O. ML estimation of time and frequency offset in OFDM systems[J]. IEEE Transactions on Signal Processing, 1997, 45(7): 1800-1805.
- [19] 姜永权, 马逸新. 一种基于 PN 序列的 OFDM 时域信道估计方法[J]. 汕头大学学报(自然科学版), 2006(4): 75-80.
- [20] 吕金花. 基于时频域联合估计的 OFDM 系统信道估计算法研究[D]. 吉林大学, 2008.
- [21] 任紫正. 基于 DTMB 信号的外辐射源雷达参考信号提纯与快速相干积累算法研究[D]. 西安电子科技大学, 2024.
- [22] 伍凌智, 王劲涛, 王军, 等. 基于 TDS-OFDM 系统的迭代信道估计与均衡方法[J]. 清华大学学报: 自然科学版, 2009(11): 1891-1895.
- [23] Lin-feng L, Jim W, Jian S. Phase Noise Correction for DTMB-A System[J]. Journal of Signal Processing, 2014, 30(7).

- [24] Zhao X, Liu P, Wang B, et al. GPU-accelerated signal processing for passive bistatic radar[J]. Remote Sensing, 2023, 15(22): 5421.
- [25] Zaknoun K, Leppänen V, Lähdekorpi P. Real-Time, GPU-accelerated Processing of Digital Radar Signal Data[J]. 2024.
- [26] Prabhu V. Error-rate considerations for digital phase-modulation systems[J]. IEEE Transactions on Communication Technology, 1969, 17(1): 33-42.
- [27] ETSI. Digital video broadcasting (DVB); measurement guidelines for DVB systems[R]. 2001.
- [28] ITU-R. Guidelines on measurements for digital terrestrial television broadcasting systems[R]. 2016.
- [29] 方宏, 卞昕, 何昭, 等. 基于星座图设计的矢量调制误差计量方法研究[J]. 仪器仪表学报, 2013, 34(1): 128-132.

致 谢

本科四年的学习生活极大的丰富了我对电子信息相关项目的认识，并且锻炼出我的良好工作规划能力和沉着冷静的心态。在此期间有诸多为我提供帮助和支持的人，我对他们表示衷心的感谢。

对于我的学习方面，首先我需要特别感谢万显荣导师和易建新导师，两位导师为我提供了很多有效的帮助，使我受益匪浅。万老师为我提供了该毕业设计的具体方向和工作内容，并在我开发毕设内容时多次为我点明了近期工作方向和规划，使得我从对雷达一无所知到现在对被动雷达拥有基本的认识。而易老师多次给我解答了关于被动雷达方面的基础问题，解决了诸多参考信号重构仿真实现时遇到的具体细节问题。祝愿我的导师们研究进展顺利，桃李满天。

此外感谢杨凯文同学，该同学一样是在进行参考信号重构的工作，虽然他的设计是在FPGA上实现，但也能为我提供诸多在硬件上利用并行化优势加速项目运行速度的思路。同时感谢周子赫、曹恒超、阳柯、郭柏宏等同学，他们在我本科参加的学科竞赛培训期间提供了很多知识上的帮助；也感谢我的三位室友孙靖宇、樊昌鑫、袁建伟同学，他们与我共同学习和娱乐了四年，相处十分融洽和谐。我的同学和室友们都各有出路，祝愿他们前程似锦。

另外我还要感谢我的家人。我的父母在我就读期间提供了丰富的物质和精神上的支持，对我进行无微不至的关心，为我的就读环境提供了充分的保障。我的爷爷奶奶也一直关心我的学业与生活，在此对我的父母和爷爷奶奶表示衷心的感谢，愿他们长寿而健康。

我还要感谢我的女朋友李思宇，她在我学习期间给予了很多鼓励与陪伴。同时，她的出现让我就读期间的生活更加丰富多彩，至今一年半的陪伴弥足珍贵。

最后，再次对我的老师、同学、家人与朋友一并表示感谢！

